

一、Python简介

- python定义

起源于1989年，发行于1991年。

免费、开源、跨平台、动态、面向对象的编程语言

- python解释器下载官网：<http://www.python.org>

- Python程序的执行方式

- 交互式

在命令行输入指令，回车即可得到结果。

1. 打开终端shell
2. 进入交互式: `python3`
3. 编写代码: `print("hello world")`
4. 离开交互式: `exit()`

- 文件式

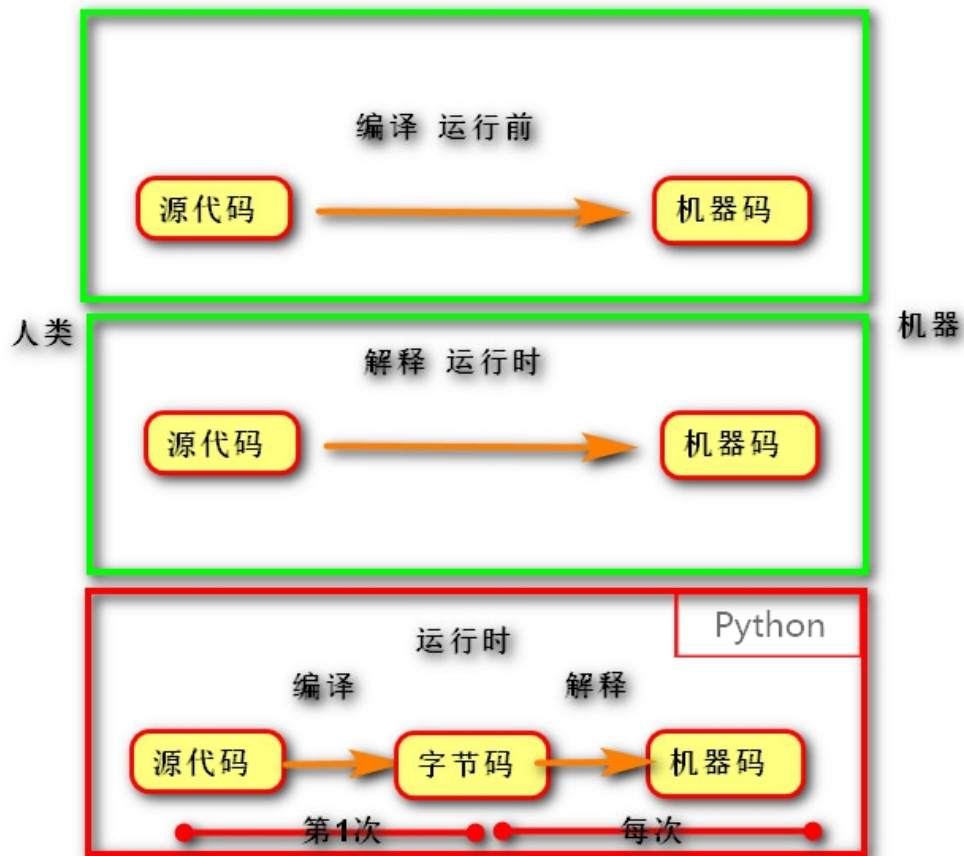
将指令编写到.py文件，可以重复运行程序。

1. 编写文件。
2. 打开终端
3. 进入程序所在目录: `cd 目录`
4. 执行程序: `python3 文件名`

- 程序执行过程

- \1. 由源代码转变成机器码的过程分成两类：编译和解释。
- \2. 编译：在程序运行之前，通过编译器将源代码变成机器码，例如：C语言。
 - 优点：运行速度快
 - 缺点：开发效率低，不能跨平台。
- \3. 解释：在程序运行之时，通过解释器对程序逐行翻译，然后执行。例如Javascript
 - 优点：开发效率高，可以跨平台；
 - 缺点：运行速度慢。
- \4. python是解释型语言，但为了提高运行速度，使用了一种编译的方法。编译之后得到pyc文件，存储了字节码（特定于Python的表现形式，不是机器码）。

源代码 -- 编译 --> 字节码 -- 解释 --> 机器码



集成开发工具 --vscode

- 环境搭建
 -
 - 配置和插件
 - 修改插件存放位置,
右击 vscode 快捷方式-属性-目标-添加下列代码
- ```
--extensions-dir 文件绝对路径 (D:\appdate\vscode\vscodeplugin)
```
- python 插件 ---代码提示
- 虚拟环境
  - pip install virtualenv
  - python -m venv 虚拟环境名称
  - ct + shift + p 进入虚拟环境 (& E:\компьютер\python\python基本知识\exercise\venv\Scripts\Activate.ps1)
  - 激活虚拟环境
    1. 在命令行中输入命令 `cd .\myvenv\Scripts` , 进入虚拟环境目录;
    2. 在命令行中输入命令 `.\Activate.ps1` , 激活当前虚拟环境;
    3. 在命令行前出现虚拟环境名称 (myvenv) 即为激活成功。

- 退出虚拟环境

deactivate

```
#linux
sudo apt-get install python3-venv #安装虚拟环境工具virtualenv
python3 -m venv venv #创建虚拟环境
source venv/bin/activate #激活虚拟环境
```

- 快捷键

| 快捷键          |       |
|--------------|-------|
| ctrl + d     | 复制一行  |
| ctrl +alt +L | 格式化代码 |
|              |       |

- 调试 (debug)

- 目的

- 1.审查程序运行时变量取值
- 2.审查程序运行的流程

- 步骤

- 1.加断点
- 2.调试运行
- 3.执行一行
- 4.停止

## 二、数据基本运算

### (一) 注释

- 1.单行注释

```
以#号开头
```

- 2.多行开头

```
"""以三个双引号或单引号"""
```

### (二) 变量与常量

- 1.变量

- 语法

```
变量名称 = 对象
变量名1 = 变量名2 = 数据
变量名1, 变量名2, = 数据1, 数据2
```

```

#变量名--语义--真实内存地址的别名
#赋值号（ = ）将右边对象的地址复制给左边内存空间
a = "对象" #将对象的内存地址给变量a
a = b = "对象", #将对象的内存地址给变量a和b
a,b = "a","b", #将'a','b'分别给变量a和b
a,b = b,a #交换对象内容
"""

a = 1
b = 2
temp = a
a = b #2
b= temp #1
"""

```

#### .命名规则

- 数字、字母和下划线"\_"组成
- 必须是字母或下划线开头，后跟字母、数字、下划线
- 严格区分大小写
- 禁止使用关键字

#### .命名规范 ---顾名思义

1. 小驼峰命名法：第一个单词的首字母小写，其余单词的首字母大写
2. 大驼峰法：每个单词的首字母大写
3. 使用下划线链接：user\_name
4. 注：在Python里的变量、函数和模块名使用下划线连接，类名使用大驼峰命名法。

#### 2.常量

python中没有常量的概念，但在编程需要有常量。变量名所有字母大写代表常量。

```
PI = 3.1415926
```

## (三) 数据基本类型

在python中变量没有类型，但关联的对象有类型

通过type()函数可查看

#### 1.空值对象None

表示不存在的特殊对象，占位和解除与对象的关联

#### 2.整形 int

- 表现形式

| 进制            | 表示              |
|---------------|-----------------|
| 二进制 bin (0,1) | 0b 开头 0b0001110 |
| 八进制 oct (0-7) | 0o开头            |

| 进制                | 表示   |
|-------------------|------|
| 16进制 hex(0-9,a-f) | 0x开头 |

- 小整数对象池  
cpython中整数-5至265永远存在小整数对象池中不会被释放，并可重复使用
- 3.浮点数 float
- 表现形式  
小数  
科学计数法 e/E(正负号) 1.23e(10)2 =123
- 4.字符串 str  
用"" 或 ' '表示
- 5.bool性  
True 或 False
- 6.数据类型变换

```
str (str())<-> int (int(str))
float (float())<-> int (int(float))
```

(四) 运算符

| 运算符   |                                         |
|-------|-----------------------------------------|
| 算术运算符 | + - *乘 / 除 // 取整 % 取余,** 幂运算(= pow()函数) |
| 增强运算符 | += 自增 -= *= /= **= //= %=               |
| 逻辑运算符 | 与 and 或 or 非 not                        |
| 比较运算符 | <,>,>=,<=,<>,! =,==                     |
| 身份运算符 | is is not (内存地址是否一致)                    |
| 成员运算符 | in not in                               |

```
#交叉赋值
m = 15
n = 56
m,n = n,m
#解压赋值
lists = [1,2,3,4]
list0,list1,list2,list3=lists
"{}{}{}{}{}".format(*lists)
```

备注

逻辑与的规则：只要有一个运算符是false，结果为false；只有所有的运算符是true，结果才是true.

逻辑或的规则：只有一个运算符为true，结果为true；只有所有的运算符是false，结果才是false。

逻辑非规则：true < (转换) > false

逻辑运算符优先级：not>and>or (括号的优先级最高，(4>5and(not 4>3))。)

is 用于判断两个对象是否是同一对象，是时返回True,否则返回False。

## 短路逻辑

#问题：控制台出现了什么？

#短路逻辑：逻辑运算时，尽量将复杂（耗时）的判断放在后边

```
num = 1
```

#and 发现 false，就有了结论，后续条件不再判断。

```
re = num > 1 and input () == ' a'
```

#or 发现true，就有结论，后续条件不再判断。

```
re = num +1 > 1 or input () =='a'
```

## (五) 数值运算函数

| 函数           | 说明          |
|--------------|-------------|
| abs(x)       | 求绝对值        |
| divmod(x,y)  | 即(x//y,x%y) |
| pow(x,y[,z]) | 即x**y %z    |
| round(x[,n]) | 使x保留n位小数    |
| max()        | 求最大值        |
| min()        | 求最小值        |

## 三、容器类型

### (一) 容器通用操作

#### 1.运算

| 类型    |                         |
|-------|-------------------------|
| 拼接    | x+y 连接两个容器 累加x +=y      |
| 复制    | x * n , x *= n 将容器x复制n遍 |
| 比较    | 对容器里的内容先转为对应的编码，再依次比较   |
| 成员运算符 | in, not in 判断一个成员是否在容器内 |

#### 2.索引

```
#语法: <变量>[]
lists = [1,2,3]
lists[0]
"""
字符串的序号

-3 -2 -1 反向

刘 亦 菲

0 1 2 正向
"""
```

注: ( <>[ ] = 可以修改容器内内容, 字符串是不可变的数据类型, 因此不可修改)



### 3.切片

```
#语法: <变量>[login:end] 返回一段内容, 范围[login,end)
lists = [1,2,2]
lists = [0:1] # 1
#高级切片 <变量>[m:n:step] 根据步长切片, 步长的正负需与mn保持一致。可以省略m或n, 表示至开头或
结尾。[: : -1]==>逆序
```

### 4.函数、方法

| 名称    |            |
|-------|------------|
| len() | 返回序列 的长度   |
| max() | 返回序列的最大值元素 |
| min() | 返回序列的最小值元素 |
| sum() | 对数值求和      |

## (二) 字符串 str

### 1. 定义

由一列字符组成的不可变序列容器, 存储的是字符的编码值

### 2. 表示

‘’, “”, """ """ 三引号可以所见即所得 str()

### 3. 字节串 (bytes)

在python3中引入了字节串的概念, 与str不同, 字节串以字节序列值表达数据, 更方便用来处理二进制数据。因此在python3中字节串是常见的二进制数据展现方式。

- 普通的ascii编码字符串可以在前面加b转换为字节串, 例如: b'hello'
- 字符串转换为字节串方法: str.encode()
- 字节串转换为字符串方法: bytes.decode()

4. 字符串格式化

对 \ 后面的字符进行转义

| 转义字符    |                         |
|---------|-------------------------|
| \ '     | 显示一个普通的单引号              |
| \ "     | 显示一个的双引号                |
| \n      | 表示一个换行                  |
| \t      | 表示一个tab键                |
| \(r''') | 表示一个普通的反斜线\             |
| \r      | 返回值开始位置 (具体用法见“库”中的进度条) |

注

取消转义

```
url = " c: \ \n\r\aoode"
```

```
url = r" c:\n\r\aoode"
```

4. 字符串格式化

- 使用%占位符

| 占位符   |               |
|-------|---------------|
| %s    | 表示字符串的占位符     |
| %%    | 输出百分数     50% |
| %d    | 表示整数的占位符      |
| %f    | 表示浮点数的占位符     |
| %.nd  | 显示n位数字，不足空格补齐 |
| %.0nd | 不足0补齐         |
| %.-nd | 表示在后面补齐       |
| %.nf  | 保留n位小数        |
| %x    | 表示以十六进制输出     |

```
print("名字:%s, 性别:%s"("张飞", "男"))
print("名字:%(name)s, 性别:%(sex)s"%{"name": "张飞", "sex": "男"}) #以字典的形式
```

- format格式



```

#"{:<填充>|<对齐>|<宽度>|<,>|<精度>|<类型>}".format()
"{ } : 计算机{ } 的CPU功率{ } ".format("2018", "1", "2")
"我是{name}, 我今年{age} ".format(age=18, name="")
"我是{name}, 我今年{age} ".format({"age":18, "name":""})

```

|      |                                         |
|------|-----------------------------------------|
|      |                                         |
| :    | 引号符号                                    |
| <填充> | 用于填充的字符(宽度不足)                           |
| <对齐> | "<" 左对齐 "^" 居中 ">" 右对                   |
| <宽度> | 设定输出字符串的宽度                              |
| <,>  | 数字的千分位分隔符                               |
| <精度> | 浮点数精度                                   |
| <类型> | 整数 b二进制, d(10) o(8) x X(16) 浮点数 e E f % |

- f格式(3.6版以后)

```

#语法: f"{ }"
age=18
name=""
f"我是{name}, 我今年{age}"
#2. 具有eval功能, 将字符串当成表达式运行
f'{print("")}'
result = f'{3+5}'

```

## 5. 编码

- \1. 字节byte: 计算机最小存储单位, 等于8 位bit.
- \2. 字符: 单个的数字, 文字与符号。
- \3. 字符集(码表): 存储字符与二进制序列的对应关系。
- \4. 编码: 将字符转换为对应的二进制序列的过程。
- \5. 解码: 将二进制序列转换为对应的字符的过程。
- \6. 编码方式:
  - ASCII编码: 包含英文、数字等字符, 每个字符1个字节。
  - GBK编码: 兼容ASCII编码, 包含21003个中文; 英文1个字节, 汉字2个字节。
  - Unicode字符集: 国际统一编码, 旧字符集每个字符2字节, 新字符集4字节。
  - UTF-8编码: Unicode的存储与传输方式, 英文1字节, 中文3字节。

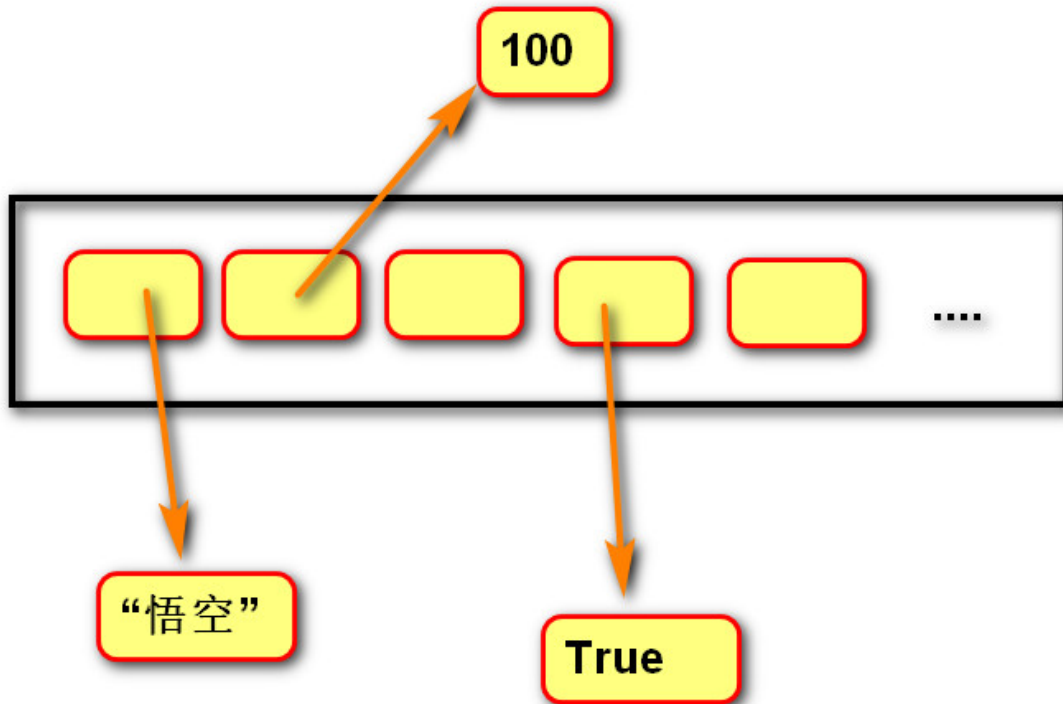
## 6. 函数

| 类型 | 函数/方法("".)                                 |                                                                            |
|----|--------------------------------------------|----------------------------------------------------------------------------|
| 其他 | ord(字符串)                                   | 返回该字符串的unicode码                                                            |
|    | chr(unicode码)                              | 返回相应的字符串                                                                   |
| 判断 | .isspace()                                 | 如果字符串中只包含空白，则返回 True，否则返回 False.                                           |
|    | .lower()                                   | 判断字符串是否是小写                                                                 |
|    | .upper()                                   | 判断字符串是否是大写                                                                 |
|    | .istitle()                                 | 判断字符串每个单词首字母是否大写                                                           |
|    | .isalnum()                                 | 判断字符串是否由字母或数字组成                                                            |
|    | .isspace()                                 | 字符串由字母组成结果为True                                                            |
|    | .startswith(substr, beg=0,end=len(string)) | 检查字符串是否是以指定子字符串 substr 开头，是则返回 True，否则返回 False。如果beg 和 end 指定值，则在指定范围内检查。  |
|    | .endswith(suffix, beg=0, end=len(string))  | 检查字符串是否以 obj 结束，如果beg 或者 end 指定则检查指定的范围内是否以 obj 结束，如果是，返回 True,否则返回 False. |
|    | .isdigit()                                 | 判断字符串是否由数字组成                                                               |
| 查找 | .find(str, beg=0 end=len(string))          | 检测 str 是否包含在字符串中，如果指定范围 beg 和 end ，则检查是否包含在指定范围内，如果包含返回开始的索引值，否则返回-1       |
|    | .rfind(str, beg=0,end=len(string))         | 从右边开始查找.                                                                   |
|    | .count(str, beg=0,end=len(string))         | 返回 str 在 string 里面出现的次数，如果 beg 或者 end 指定则返回指定范围内 str 出现的次数                 |
| 修改 | .replace(old, new [, max])                 | 把 将字符串中的 str1 替换成 str2,如果 max 指定，则替换不超过 max 次。                             |
|    | .lstrip()                                  | 截掉字符串左边的空格或指定字符。                                                           |
|    | .rstrip()                                  | 删除字符串字符串末尾的空格.                                                             |
|    | .strip([chars])                            | 在字符串上执行 lstrip()和 rstrip()                                                 |
|    | .lower()                                   | 转换字符串中所有大写字符为小写.                                                           |
|    | upper()                                    | 转换字符串中的小写字母为大写                                                             |
|    | swapcase()                                 | 将字符串中大写转换为小写，小写转换为大写                                                       |

### (三) 列表 list

#### 1.定义

由一系列变量组成的可变序列容器



列表内存图

#### 2.表示

[<对象>,] 或 list(<其他容器>)

#### 3.操作

- 获取元素

```
#索引
lists = [1,2,3]
lists[0]
#切片 通过切片获取元素，会创建新列表，仅仅拷贝第一层 -----浅拷贝
list[0:1]
```

- 修改

```
#语法: list.[index/切片] =
lists = ["刘亦菲","貂蝉"]
lists[1] = "杨幂"
lists[1] =["杨幂","刘诗诗"]

#L.sort(reverse=False) | 将列表中的元素进行排序，默认顺序按值的小到大的顺序排列
#L.reverse() | 列表的反转，用来改变原列表的先后顺序 进行排序，容器内的元素必须是同种类型
```

- 增加

| 方法                    |            |
|-----------------------|------------|
| .append(object)       | 在列表末尾添加    |
| .insert(index,object) | 在index位置添加 |

- 合并列表

#语法: `list_a.extend(list_b)` 把b追加到a里

- 删除

| 方法                           |                      |
|------------------------------|----------------------|
| .pop([index]) / del a[index] | 默认(没有index), 删除末尾元素  |
| .remove(object)              | 删除列表里的内容, 若不存在, 则报错。 |
| .clear()                     | 清楚列表                 |

注:

删除实质: 在内存中, 后一个替换前一个

-----  
可以从后往前删

for i in range (-(len(a),-1,-1):

#### 4.遍历列表

```
#语法: for item in list:
lists = ["1","2",1]
for item in lists:
 print(item)
#倒叙
#1) for i in list02[::-1]:
#list02[::-1] 通过切片拿元素, 会重新创建新列表。浪费内存, 不建议
(2) 0 1 2 3 => 3 2 1 0 或 -1 -2 -3 -4
for i in range(len(lists)-1, -1 , -1) :
 print(lists[i])
for i in range (-1,-len(lists)-1,-1):
 print(lists[i])
```

#### 5.拷贝

- 浅拷贝

通过切片, [ 4,[ a,b]]只拷贝一层,内面容器的内容不拷贝

a.copy()

- 深拷贝

将所有内容拷贝, 使用copy模块, a=copy.deepcopy(b)

- 注：
- 不过是浅拷贝还是深拷贝，都无法拷贝不可变的容器

## 6.列表推导式

- 定义  
使用简易方法，将可迭代对象转换成列表
- 语法

```
#变量 = [表达式 for 变量 in 可迭代对象]
#变量 = [表达式 for 变量 in 可迭代对象 for 变量 in 可迭代对象] 双for结构
#变量 = [表达式 for 变量 in 可迭代对象 if 条件]
```

## 7.列表扩容

- 1.列表在创建时都会预留空间
- 2.当预留空间不足时，都会在创建新列表（更大的空间）
- 3.将原有数据拷贝到新列表中
- 4.替换引用

## (四) 元组 tuple

### 1.定义

由一系列变量组成的不可变序列容器，一旦创建，里面的变量不可操作（按需分配）

### 2.操作

- 创建

```
#空元组
a= ()
a = tuple()
#默认值
a = (1,2)
#元组只有一个元组
a = (1,)
```

- 获取元素  
同列表，索引、切片

## (五) 字典

### 1.定义

由一系列键值对组成的可变映射容器

映射：一对一的对应关系

键必须唯一且不可变（字符串/数字/元组），值没有限制

### 2.操作

- 创建

```

#空字典
dict0 = {}
dict0 = dict()
#默认值
dict01 = {"1":1}

dict01 = {x=1,y=1}

dict01 = dict(<容器>) #dict([("a","1"),("age",12)])

keys = ["name","age"]
value = None
dict01 = {}.formkeys(keys,values<默认值>)
#
list1 = ["林妹妹","宝姐姐"]
list2 = [101,102]
dict1 = dict(list(zip(list1,list2))) #zip,将多个列表的每个元素合并成元组

```

- 查找

```

dict01 = {" name":"杨幂","sex":"女"}
#1.dict[key]
dict01["name"] #杨幂
#2.dict.get(key,value)
print(dict01.get("name","刘亦菲")) #若字典中不存在，则返回"刘亦菲"

```

- 修改/添加

```

#dict[key] = 若字典中存在，则修改，否则是增加
dict01["name"] = "张飞" #dict01 = {" name":"张飞","sex":"女"}

```

- 删除

```

#语法: del dict01[]

```

- 遍历

```

"""
for key,value in dict01.items(): 得到key,value
for i in dict01.items(): 以元组形式得到键值
for i in dict01: 得到key
for i in dict01.values(): 得到value
"""

```

### 3.列表与字典的嵌套

- 字典内嵌列表 {<>:[ ]}
- 字典内嵌字典 {<>:{}}
- 列表内嵌字典 [ { : } ]
- 列表内嵌列表

#1. exercise05字典内嵌列表:

```
{
 "张无忌": [28, 100, "男"],
}
#2. exercise06字典内嵌字典:
{
 "张无忌": {"age": 28, "score": 100, "sex": "男"},
}
#3. exercise07列表内嵌字典:
[
 {"name": "张无忌", "age": 28, "score": 100, "sex": "男"},
]
#4. 列表内嵌列表
[
 ["张无忌", 28, 100男],
]
"""
```

选择策略：根据具体需求，结合优缺点，综合考虑（两害相权取其轻）。

字典：

优点：根据键获取值，读取速度快；

代码可读性相对列表更高（根据键获取与根据索引获取）。

缺点：内存占用大；

获取值只能根据键，不灵活。

列表：

优点：根据索引/切片，获取元素更灵活。

相比字典占内存更小。

缺点：通过索引获取，如果信息较多，可读性不高。

"""

#### 4.推导式

```
#语法: { item:item ** 2 for item in range(1,11) if item >5}
```

#### 5.函数

- 查找

| 函数/方法                          |                                        |
|--------------------------------|----------------------------------------|
| .get(key, default=None)        | 返回指定键的值，如果值不在字典中返回default值             |
| .setdefault(key, default=None) | 和get()类似，但如果键不存在于字典中，将会添加键并将值设为default |
| .popitem()                     | 随机返回并删除字典中的一对键和值(一般删除末尾对)。             |
| .items()                       | 以列表返回可遍历的(键, 值) 元组数组                   |
| .keys()                        | 返回一个迭代器，可以使用 list() 来转换为列表             |
| values()                       | 返回一个迭代器，可以使用 list() 来转换为列表             |

- 修改

| 方法            |           |
|---------------|-----------|
| update(dict2) | 字典记录累加    |
| clear()       | 删除字典内所有元素 |

## 6.键值互换

```
#语法
dic = { value:key for key ,value in dict01.items()}
#value重复时
li = [(value,key) for key,value in dict01.items()]
```

## (六) 集合 set

### 1.定义

由一系列不重复的不可变类型变量组成的可变映射容器，相当于有键无值的字典

### 2.操作

- 创建

```
#空值
set0 = set()
set1 =set(可迭代对象)
#默认值
set2 = {1,2,3}
```

- 添加 .add( <元素>)
- 删除 .discard( <元素>)
- 遍历 for item in set0:

### 3.函数

[update\(\)](#) 给集合添加元素/其他集合

[clear\(\)](#) 移除集合中的所有元素

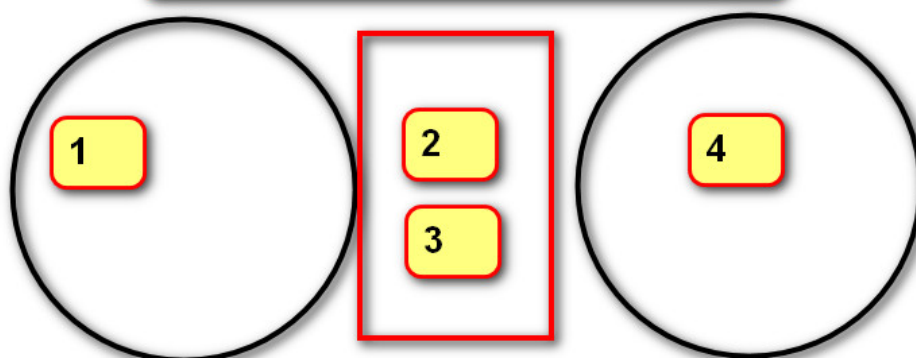
[copy\(\)](#) 拷贝一个集合

[pop\(\)](#) 随机移除元素

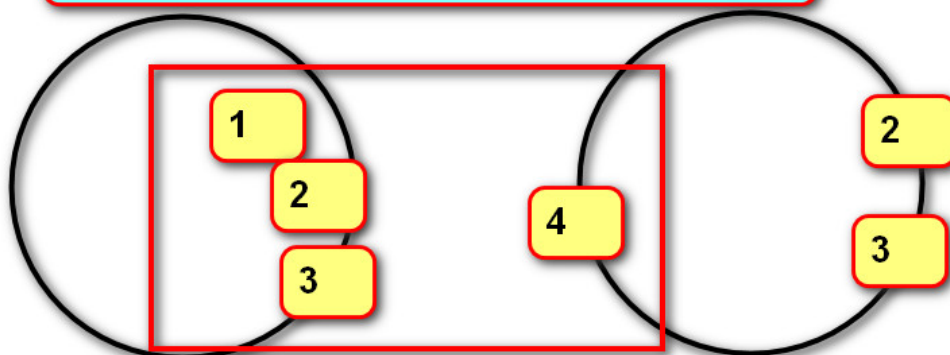
### 4.运算



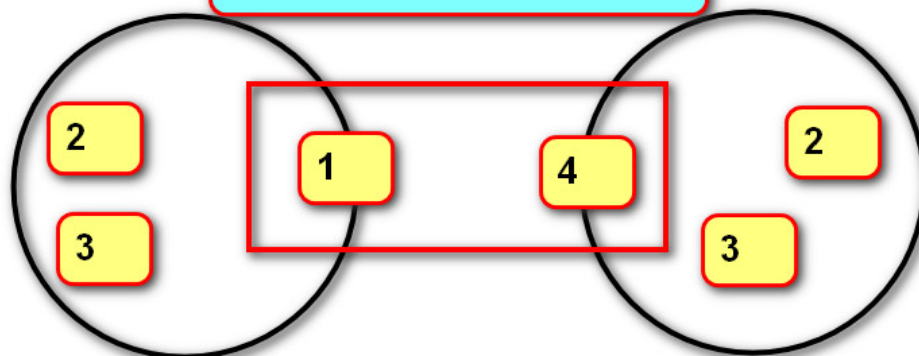
交集：返回两个集合中相同元素。



并集：返回两个集合中所有不重复的元素。



补集：返回不同的元素。



交集 &

并集 |

补集 (两边不同) ^

补集 (一边不同) -

子集 <

超集 >

5.固定集合

不可变集合 -frozenset()

运算同集合

## (七) 容器转换

| 列表 => 字符串 | result="连接符".join() 注：列表内的元素须是字符串 |
|-----------|-----------------------------------|
| 字符串 => 列表 | result="a-b-c".split(<分隔符>)       |
| 列表 => 元组  | tuple( )                          |
| 元组 => 列表  | list( )                           |
| 集合=>字符串   | 先转为列表，再转                          |

- 列表=> 字符串

```
#缺点：每次循环形成一个新的字符串对象，替换变量引用result
result = ''
for item in range(10):
 #""
 #"0"
 #"01"
 result += str(item)

#优点 每次循环只向列表添加字符串，并没有创建列表对象
result = []
for item in range(10):
 result.append(str(item))

print("".join(result))
```

## 四、语句

### (一) 选择语句 (if,elif,else)

1. 作用:

让程序根据条件选择性的执行语句。

2. 语法:

if 条件1:

    语句块1

elif 条件2:

    语句块2

else:

    语句块3

注:

elif 子句可以有0个或多个。

else 子句可以有0个或1个，且只能放在if语句的最后。

```
#猜数字游戏
import random
int_num = random.randint(0,100) #随机产生[0,100]之间的数
count = 0 #统计次数
while True:
 int_input = int(input("请输入数字"))
 if int_input > int_num:
 print("您输入的数字太大了")
 count += 1
 elif int_input < int_num:
 print("您输入的数字太小了")
 count += 1
 else:
 count += 1
 print("您猜对了,猜了{}次".format(count)) #猜对，停止循环
 break
```

2. 条件表达式

```
sex = None

if input("请输入性别") == "男":

 sex = 1

else:

 sex = 0

print(sex)
```

```

#条件表达式
```

```
sex = 1 if ("请输入性别") == "男" else 0
```

```
print(sex)
```

## (二) 循环语句

### 1.while语句

```
#语法:
```

```
"""
```

```
1.while <条件>: 死循环 while True:
```

```
2.while <>: while的条件不满足,才执行else
```

```
 else:
```

```
"""
```

注:

else子句可以省略。

在循环体内用break终止循环时,else子句不执行。

```
#斐波那契数列 0 1 1 2 3 5 8
```

```
def get_list(num):
```

```
 count = 0
```

```
 int_num1 = 0
```

```
 int_num2 = 1
```

```
 lists=[]
```

```
 while count < num:
```

```
 lists.append(int_num1) #将int_num1存进列表
```

```
 temp = int_num1 #暂时存储
```

```
 int_num1 = int_num2 #将int_num2的值赋值给int_num1
```

```
 int_num2 = temp+int_num1
```

```
 count += 1
```

```
 return lists
```

```
def get_list(num):
```

```
 int_num1 = 0
```

```
 int_num2 = 1
```

```
 lists=[]
```

```
 for i in range(num):
```

```
 lists.append(int_num1) #将int_num1存进列表
```

```
 temp = int_num1 #暂时存储
```

```
 int_num1 = int_num2 #将int_num2的值赋值给int_num1
```

```
 int_num2 = temp+int_num1
```

```
 return lists
```

```
get_list(10)
```

## 2.for循环

```
"""
语法
1.for 变量 in 可迭代对象:
 循环体

2.for 变量 in range (): range(开始值, 结束值, 间隔) - > 整数生成器

3.for <> :
 else:

4.for item in enumerate(<迭代对象>): item => index,element (索引, 元素)

5.for item in zip(<迭代对象>,<迭代对象>):
 item => (元素, 元素)
"""
```

注：双for循环 -> 外层循环控制行，外层循环控制列

```
#乘法口诀
for i in range(1,10):#外循环控制行
 for j in range(1,i+1):#内循环控制列
 print(f"{j}*{i}={i*j}",end=" ")
 print() #换行
```

## 3.跳转语句

- break 满足条件，则退出本层循环
- continue 满足条件，则跳过本次循环，继续下次循环

## (三) pass语句

用来占位

```
for item in list: #pass语句可以当占位符，
 pass
```

## (四) del语句

```
语法
del a 删除变量a,同时解除与对象的关联
```

## (五) 异常处理

### 1.定义

当异常发生时，程序不会再向下执行，而转到函数的调用

### 2.类型

名称异常(NameError):变量未定义。.  
类型异常(TypeError):不同类型数据进行运算  
索引异常(IndexError):超出索引范围  
属性异常(AttributeError):对象没有对应名称的属性  
属键异常(KeyError):没有对应名称的键  
为实现异常(NotImplementedError):尚未实现的方法.  
异常基类Exception。

```
try : .可能触发异常的语句
except 错误类型1 [as变量1]:.
 处理语句1.
except错误类型2 [as变量2]:.
 处理语句 2.
except Exception [as变量3]:.
 不是以上错误类型的处理语句.
else : .
 未发生异常的语句
finally :
 无论是否发生异常一定会执行的语句
```

-----

as 子句是用于绑定错误对象的变量，可以省略  
except子句可以有一个或多个，用来捕获某种类型的  
else子句最多只能有一个。.  
finally子句最多只能有一个，如果没有except子句如果异常没有被捕获到，会向上层(调用处)继续传递，

#自定义异常

1.定义:

```
class 类名Error(Exception) :.
 def__init__ (self,参数):.
 super (). _init_(参数). #参数，错误信息（错误编号，内容等）
 self.数据=参数。
```

2.调用:。

```
try : .
 #raise 自定义异常类名(参数).
except定义异常类as变量名:,
 变量名.数据。
```

## (六) raise语句

作用

抛出一个错误，让程序进入异常状态

```
#语法: raise 异常种类
if type(eval(input("处理列表"))) == list:
 print()
else:
 raise ValueError()
```

# 五、函数

表示一个功能，函数定义者是提供功能的人，函数调用者是使用功能的人

## (一) 内置函数

### 1.基本函数

| 函数          | 备注 | 说明          |
|-------------|----|-------------|
| print(end=) |    | 输出函数        |
| input ( )   |    | 输入函数        |
| type(a)     |    | 查看a的数据类型    |
| id()        |    | 获取变量存储的对象地址 |

### 2.数据类型转换函数

| 函数                | 说明                       |
|-------------------|--------------------------|
| int(<对象>[, base]) | 转化为整数，base-进制转换（2，8，,16） |
| float(<对象>)       | 转化为浮点数                   |
| str(<对象>)         | 转换为字符串                   |
| bool(<对象>)        | 转化为布尔值                   |

注：

- 1.数字0，空字符串、None、空列表、空字典、空元组布尔值是False，其他是True
- 2.在计算机里，True 和False使用数字1和0表示

## (二) 高阶函数

### 1.定义

将函数作为参数或返回值的函数

### 2.内置函数

| 函数                         |                            |
|----------------------------|----------------------------|
| map(<函数>,<迭代对象>)           | 映射 == select,返回新的可迭代对象     |
| filter(函数,可迭代对象)           | 根据条件筛选可迭代对象中的元素，返回新的可迭代对象。 |
| sorted(可迭代对象，key =函数，bool) | 排序                         |
| max(可迭代对象，key =函数)         |                            |

|                            |  |
|----------------------------|--|
| 函数                         |  |
| min(可迭代对象,key =函数(lambda)) |  |

```
lists = [4,10,6,20,50,45]
for item in max(lists,key=lambda item:item):
 print(item)
```

## (三) 自定义函数

设计思想---分而治之

### 1.定义函数

```
"""
语法:

def <函数名>(<形式参数>):
 """文字说明功能 """
 函数体
"""

def print_hello(): #无参函数
 """
 打印hello
 """
 print("hello")

#有参函数
def print_num(n):
 """
 1-n的和
 """
 result = 0
 for i in range(101):
 result += i
 return result

#空函数 用于构思代码
def func(x):
 pass
```

可变 / 不可变类型在传参时的区别

1. 不可变类型参数有:

数值型(整数, 浮点数,复数)

布尔值bool

None 空值

字符串str

元组tuple



固定集合frozenset

2. 可变类型参数有:

列表 list

字典 dict

集合 set

3. 传参说明:

不可变类型的数据传参时，函数内部不会改变原数据的值。

可变类型的数据传参时，函数内部可以改变原数据。

## 2.调用函数

```
#语法: <函数名>(<实际参数>)
"""
 *

 """
def print_diagram():
 for i in range(1,8,2):
 j = int((7-i)/2)
 print(" "*j+"*"*i+" "*j)

print_diagram()#调用函数
```

3.返回值 return

```
#作用: 返回结果，退出函数
def a(i):
 return i
print(a(1)) #1
```

满足以下两个条件，就无需通过返回值传递结果。

传入的是可变对象

函数体修改的是传入的对象

## 4.作用域 LEGB

- 类型

- \1. Local局部作用域：函数内部。
- \2. Enclosing 外部嵌套作用域：函数嵌套。
- \3. Global全局作用域：模块(.py文件)内部。
- \4. Builtin内置模块作用域：builtins.py文件。

- 变量查询规则

L => E => G => B

- 函数内修改全局变量

```
g01 = "ok"

def fun01():
 a = 0 #局部变量，只能在本函数内使用
 global g01

 g01 = "no"

 global g02 #定义一个全局变量

 g02 = "20"

 print(a) #a是局部变量，会报错
```

## 5.函数参数

- 实际参数

```
#1. 位置实参----实参与形参的位置依次对应
def fun01(a,b,c,d):
 print(a,b,c,d)

fun01(1,2,3,4)

#2. 关键字实参----实参与形参的根据名称进行对应
def fun01(a,b,c,d):
 print(a,b,c,d)

fun01(b=1,d=2,a=3,c=4)

#3. 序列实参
def fun01(a,b,c,d):
 print(a,b,c,d)

list01 = [1,2,3,4]
fun01(*list01) #星号将序列拆分后按位置与形参进行对应

#4. 字典实参
def fun01(a,b,c,d):
 print(a,b,c,d)

dict01 = {"a":1,"b":2,"c":3,"d":4}
fun01(**dict01) #**将字典拆分后按名称与形参进行对应
```

- 形式参数

```
#1. 缺省参数(默认) 若实参不提供，可以使用默认值
def fun01(a=0,b=0,c=0,d=0):
 print(a,b,c,d)

fun01(b=2,c=3) #与关键字实参：调用者可以随意传递参数

#2. 位置形参 注
def fun01(a,b,c): #a:int 可以限制参数的数据类型
 pass
```

```
fun01(a,b,c)
```

#3. 星号元组形参

```
def fun01(*args) : #*args将所有实参合并为一个元组，让实参个数无限
 print(a)
```

```
fun01(a)
```

#4. 命名关键字形参 在星号元组形参以后的位置形参

```
def fun01(*args,a,b) :

 print(a,args,b)
```

```
fun01(1,2,3,b=2)
```

```
fun01(a=1,b=2)
```

#作用：强制使用关键字传参

#5. 字典形参 实参可以传递数量无限的关键字实参

```
ef fun01(**kwargs) : **将所有实参合并为一个字典，让实参个数无限
 print(a)
```

```
fun01(a=1,b=2)
```

#fun01(\*\*{ }) 拆字典传参

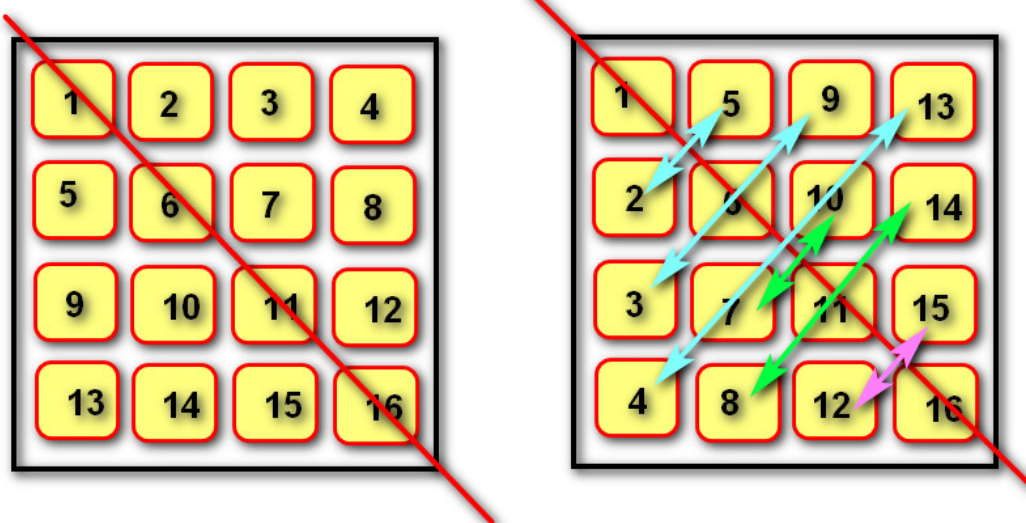
注：组合使用 位置形参，默认形参，\*args，命名关键字形参，\*\*kwargs

```
def func(x,y=1,*args,z,**kwargs):
 pass
```

函数内存图



矩阵装置



```
lists = [
 [1,2,3,4],
 [4,5,6,7],
 [8,9,10,11],
```

```

[12,13,14,15]
]

for i in range(len(lists)):
 for j in range(i+1,len(lists)):
 lists[i][j],lists[j][i] =lists[j][i],lists[i][j]

print(lists)

"""
[[1, 4, 8, 12], [2, 5, 9, 13], [3, 6, 10, 14], [4, 7, 11, 15]]
"""

```

6.名称空间，存放名字的地方，是对栈区的划分

作用：有了名称空间，就可以在栈区中存放相同的名字，划分为3种：

(1) 内置名称空间

存放的名字：存放python解释器内置的名字

`print input` 等

存放周期：python解释器启动则产生，解释器关闭则销毁

(2) 全局名称空间

非内置的名字以及非函数内定义的名字

存活周期：python文件执行则产生，文件结束则销毁

(3) 局部名称空间

函数内定义的名字

存活周期：在调用函数时存活，函数结束则销毁

```

a = 6 #全局
c= 2
print(a) #a= 6

def func():
 a= 5 #局部
 print(a) #a=5 采用就近原则
func()
print(a) #a= 6

```

## (四) 函数式编程

用一系列函数解决问题——函数作为参数

Python 使用 **lambda** 来创建匿名函数。

lambda 函数是一种小型、匿名的、内联函数，它可以具有任意数量的参数，但只能有一个表达式。

匿名函数不需要使用 **def** 关键字定义完整函数。

lambda 函数通常用于编写简单的、单行的函数，通常在需要函数作为参数传递的情况下使用，例如在 `map()`、`filter()`、`reduce()` 等函数中。

## lambda 语法格式:

```
lambda arguments: expression
```

- `lambda` 是 Python 的关键字，用于定义 lambda 函数。
- `arguments` 是参数列表，可以包含零个或多个参数，但必须在冒号(:)前指定。
- `expression` 是一个表达式，用于计算并返回函数的结果。

- 定义

用一系列函数解决问题

函数可以赋值给变量，赋值后变量绑定函数。

允许将函数作为参数传入另一个函数。

允许函数返回一个函数。

```
list01=[43,4,5,5,6,7,87]
def find(func,list01): #“封装” “继承” “多态”
 for item in list01:
 if func(item):
 yield item

#条件类
def func1(item):

 return item %2 == 0

def func2(item):

 return item >= 10

def func3(item):

 return 10<= item <= 50

for item in find(func1):

 print(item,end=" ")

print("-----")

#lambda练习

for item in find(lambda item:item >= 10):#lambda item:item%2==0
#lambda s,b :s>b
 print(item,end=" ")
```

## (五) 闭包

实质：函数作为返回值

逻辑连续，当内部函数被调用时，不脱离当前逻辑。

1.三要素：

必须有一个内嵌函数

内嵌函数必须引用外部函数中变量

外部函数返回值必须是内嵌函数

2.语法

```
def 外部函数名(参数):
 外部变量
 def 内部函数名(参数):
 使用外部变量
 return 内部函数名

#调用
变量 = 外部函数名
变量()

def give_money(money):
 print("得到%d钱"%money) #外部嵌套作用域
 def buy(target,price):
 nonlocal #修改外部嵌套作用域用 nonlocal
 if money >=price:
 money -= price
 else:
 print("钱不够")
 return buy

money = give_money(1000)
money("猪肉",22)
money("零食",100)
```

## (六) 装饰器

1.定义：

在不改变原函数的调用以及内部代码情况下，为其添加新功能的函数

2.语法

```

"""
def 函数装饰器名称(func):
 def wrapper(*args,**kwargs):
 需要添加的功能
 return func(*args,**kwargs)
 return wrapper

@函数装饰器名称
def func():
 函数体

func()
"""

```

## (七) 函数递归

1.函数的递归调用：是函数嵌套调用的一种特殊形式

2.递归的两个阶段：

回溯：一层层调用下去

递推：满足某种结束条件，结束递归调用，然后一层一层返回

```

#使用递归求n项和
def num_sum(n):
 if n == 1: #递归的结束条件，必须有，否则就成了死循环了
 return 1
 else:
 return num_sum(n-1) + n

num_sum(100)

#用递归将所有数字打印出来
lists = [1,2,[3,4,[5,6,7]]]
def print_num(lists):
 for item in lists:
 if type(item) == list:
 print_num(item)
 else:
 print(item)
print_num(lists)

```

```

#二分查找法 （递归）
lists = [1,2,3,4,6,8,10,13,15,16]
def get_num(num,lists):
 if bool(lists) == False:
 print("没找到")
 else:
 index = len(lists) // 2
 if num == lists[index]:
 return lists[index]
 elif num < lists[index]: #比中间的值小，
 get_num(num,lists[:index]) #查找左边的值

```

```
elif num > lists[index]: # #比中间的值大，
 get_num(num, list[index+1:]) #查找右边的值

get_num(8, lists)
```

## 六、面向对象

### (一) 面向过程

定义：分析解决问题的步骤，然后逐步实现

公式：程序=算法+数据结构

### (二) 基本内容

1.定义

找出解决问题的人，然后分配职责

2.公式

程序=对象+交互

3.思想

识别对象，找人

分配职责，干活

建立交互，调用



```
class computer:
 #数据成员
 def __init__(self, brand_name, cpu_model, color):
 #self 是调用当前方法的对象地址
 self.brand_name = brand_name
 self.cpu_model = cpu_model
 self.color = color
 #方法成员
 def print_information(self):
 """
 打印产品信息
 :return:
 """
 print("电脑品牌是{}, 颜色是: {}, cpu型号是
 {}, ".format(self.brand_name, self.color, self.cpu_model))
 def open_computer(self):
 """
 启动电脑
 :return:
 """
 print("正在开机")
#创建对象，实际在调用__init__方法
com01 = computer("戴尔", "黑色", "inter_ci5")
com01.print_information()
com01.open_computer()
```



```
com01 = computer("联想","黑色","inter_ci511")
com01.print_information()
com01.open_computer()
```

### (三) 具体内容

- 类 class -

定义：一个抽象的概念，如动物、人类

格式：类名所有单词首字母大写

——init—— 构造函数，创建对象时被调用

组成：

数据（数据不同，类可能相同）

方法（方法不通，则类不同）

- 对象

定义

类的实体

- 实例变量与实例方法

```
"""
实例变量

调用 print (对象.<变量>)
对象.<变量> = #修改

实例方法
def <>(self, 参数列表):
 方法体
调用：对象.<>
作用：操作实例的数据
"""

class Person: #类
 def __init__(self, name, age): #数据成员
 self.name = name
 self.age = age
 def eat(self): #方法
 print("{}吃了饭".format(self.name))

name = "刘亦菲"
age = 32
peo = Person(name, age) #对象 具体的事物
peo.age = 31 #修改
peo.eat()
```

注：

对象：类的具体事例，即归属于某个类别的个体，（具体的事物）

类是创建对象的模板。

--数据成员（变量）：名词类型的状态。

--方法成员（函数）：动词类型的行为

- 类变量与类方法

含义：

类变量 描述所有对象的共有数据

类方法 用于操作类变量

语法

```
"""
定义：
class <>: #在类中，方法外定义变量
 变量名 = 表达式 #数据

 @classmethod
 def <>(cls,参数列表): #方法
 方法体

调用：
类名.变量名
类名.<方法名>
"""
class Beauty:
 count =0 #类变量，统计人数
 def __init__(self,name,age):
 self.name = name
 self.age = age
 Beauty.count += 1 #类变量必须用类调用
 @classmethod
 def print_count(cls):
 print("共有%d个人"%Beauty.count)

beauty1 = Beauty("刘亦菲",32)
beauty2 =Beauty("貂蝉",19)
Beauty.print_count() #2人
```

- 静态方法

既不操作类变量，也不操作实例变量，将函数引入类中，就不需要加self

```
"""
语法
定义
@staticmethod
def <>():
 方法体

调用
类名.方法名()
"""
```

```

#二维向量
lists = [
 ["00", "01", "02", "03"],
 ["10", "11", "12", "13"],
 ["20", "21", "22", "23"],
 ["30", "31", "32", "33"],
]

class Vector:
 "坐标"
 def __init__(self, x, y):
 self.x = x
 self.y = y
 @staticmethod #静态方法
 def up(self):#上移
 return (-1, 0)
 def down(self):#下移
 return (1, 0)
 def right(self):#上移
 return (0, 1)
 def left(self):#上移
 return (0, -1)

class VectorHelper:
 def __init__(self, lists):
 self.lists = lists
 def get_elements(self, place, vect_dice, count):
 """
 place 位置坐标
 vect_dice 位移方向
 count 距离
 """
 list_temp = []
 for i in range(count):
 place.x += vect_dice.x
 place.y += vect_dice.y
 list_temp.append(self.lists[place.x][place.y])

 print(list_temp)
helper = VectorHelper(lists)
helper.get_elements(Vector(0, 2), Vector.down, 2)

```

## (四) 特性

### 1.封装

#### (1)定义

从数据角度讲，将基本数据复合成一个自定义类型

从行为讲，向类外提供必要的功能，隐藏实现的细节

从设计角度，分而治之，变则疏之(变化点独立封装)，高内聚(单任务)，低耦合

#### (2)私有成员



```
"""
定义
@property #只负责拦截读
def <name>(self):
 return self.__name
@name.setter #只负责拦截写入操作
def <name>(self,name):
 self.__name = name
调用
对象.属性 = 数据
变量 = 队形.属性名
"""
class Person:
 def __init__(self,name,age):
 self.name = name
 self.age = age

 @property
 def age(self):
 return self.__age

 @age.setter
 def age(self,age):
 if 0<=age<=100:
 self.__age = age
 else:
 raise ValueError("age的范围不对")

w01 = Person("刘亦菲",32)
w01.age = 33
```

## 2.继承

### (1)性质

子类之间可以相互调用，子类也可以调用父类成员，父类对象只可以调用父类成员

多个子类在概念上一致的，所以就抽象出一个父类；

多个子类的共性，可以提取到父类中，

在实际开发中：

从设计角度：先有子，再有父

从编码角度：先有父，再有子

价值-----父类隔离子类的变化，规范子类

### (2)组成

**变量：**

子类若没有构造函数，则使用父类的构造函数(init)；

子类若具有构造函数，则必须先调用父类

子类通过super().\_\_init\_\_()调用父类参数

子类可以调用父类的方法

方法-----子类可以调用父类的方法

```
class Person:

 def __init__(self.name):

 self.name = name

class Student(Person):

 def __init__(self,name,score):

 super().__init__(name)
 self.score = score

per = Person()
stu = Student()
#判断对象是否属于一个类型
print(isinstance(per,Person)) #True
#判断一个类型是否属于另一个类
print(issubclass(Student,Person)) #True
```

### 3.多态

定义

父类的同一种动作或者行为，在不同的子类有不同的实现  
调用父，执行子

### 4.设计原则

开-闭原则（目标）

对扩展开放，对修改关闭

增加新功能，不改变原有代码

类的单一职责（类的定义）

依赖倒置（依赖抽象）

客户端代码（调用的类，使用者）尽量（使用）抽象的组件（做一个父类-继承）。抽象的是稳定；实现是多变

组合复用原则

复用的最佳实践

里氏替代-

迪米特法则



```

#图形管理器
class GraphicalManager:
 def __init__(self):
 self.__graph_list = []
 def insert_graph(self, graph):
 if isinstance(graph, Graphical): #判断对象graph是否属于Graphical类
 self.__graph_list.append(graph)
 else:
 raise ValueError()
 def sum_area(self): #计算所有图形的面积和
 result = 0
 for item in self.__graph_list:
 result += item.calcu_area()
 return result

class Graphical: #图形
 #父类太过于抽象，无法写出方法体
 def calcu_area(self):
 #若子类不重写，则报错
 raise NotImplementedError()

class Square(Graphical):
 def __init__(self, length, wide):
 self.length = length
 self.wide = wide
 def calcu_area(self):
 return self.length * self.wide

squ=Square(4,5)
manager = GraphicalManager()
manager.insert_graph(squ)
print(manager.sum_area)

```

## 5.类与类的关系

泛化（继承）

b类继承a类

==关联（组合）==

a类中包含B类成员 作用域整个类

==依赖==

b类作为a类中方法的参数 作用域一个方法



组合复用

```

#员工管理器
class StaffManager:
 def __init__(self):
 self.__staff_lists=[] #员工列表

 def insert(self, staff): #将员工添加到列表

```

```

 if isinstance(staff, Staff):
 self.__staff_lists.append(staff)
 else:
 raise ValueError()
 def sum_salary(self): #计算员工工资和
 result = 0
 for item in self.__staff_lists:
 result += item.calcu_salary()
 return result

#员工
class Staff:
 def __init__(self, salary): #员工基础工资
 self.salary = salary
 def calcu_salary(self):
 raise NotImplementedError()

class Programmer(Staff):
 def __init__(self, salary, project_salary): #基础工资+项目分红
 super().__init__(salary)
 self.project_salary = project_salary

 def calcu_salary(self):
 return self.salary + self.project_salary

```

## 6. 常见内置函数

isinstance(<对象>, 类)      判断对象是否是一个类型，返回true, false  
 issubclass(<类>, 父类)      判断一个类型是否属于另一个类型  
 type()                      可以判断类型  
**内置可重写函数**      以双下划线开头，双下划线结尾  
 \_\_str\_\_ () 函数      将对象转化为字符串（对人友好）  
 \_\_repr\_\_ ()              将对象转化为字符串（解释器可识别）

```

class Stu:
 def __init__(self, name, sex):
 self.name = name
 self.sex = sex

 def __str__(self): #任意格式
 return "{} {}".format(name, sex)

 def __repr__(self): #有固定的格式
 return "Stu({}, {})".format(name, sex)

stu = Stu("林妹妹", "女")
print(stu)
stu1 = repr(stu) #克隆对象
eval(stu1) # stu对象 Stu("林妹妹", "女")

```

## 7. 运算符重载



### (1)定义

让自定义生成的对象能够使用运算符

### (2) 类型

算数运算符重载-----格式：对象 +数字

算数运算符重载

反向算数运算符重载 -----格式：对象 +数字

反向算数运算符重载

```
class Vector:
 def __init__(self,x):
 self.x = x

 def __add__(self, other): #算数运算符重载

 return Vector(self.x+other)

 def __sub__(self, other):

 return Vector(self.x - other)

 def __mul__(self, other):

 return Vector(self.x * other)

 def __radd__(self, other): #反向算数运算符

 return Vector(other+self.x)
```

复合运算符重载

复合运算符重载

比较运算符重载

比较运算符重载

## 七、文件操作

### (一) 打开

#### 1.格式

- <变量名> = open("<路径>\<文件名.txt>","<打开模式>")

```
txt = open(r"C:\a\a\txt.txt","x")
txt = open("txt.txt","w") #相对路径，txt文件要与py文件处于同一文件
```

打开一个文件后我们就可以通过文件对象对文件进行操作了，当操作结束后使用close（）关闭这个对象可以防止一些误操作，也可以节省资源。

```
file_object.close()
```

- with 上下文管理

python中的with语句使用于对资源进行访问的场合，保证不管处理过程中是否发生错误或者异常都会执行规定的“清理”操作，释放被访问的资源，比如有文件读写后自动关闭、线程中锁的自动获取和释放等。

with语句的语法格式如下：

```
"""
with open("<路径>\\<文件名.txt>","<打开模式>",encoding="utf-8") as <变量>[,]: #加上
with,操作完文件不需要.close,会自动关闭 ;encoding可以是其他编码
 文件操作
"""
```

通过with方法可以不用close(),因为with生成的对象在语句块结束后会自动处理，所以也就不需要close了，但是这个文件对象只能在with语句块内使用。

```
"""
题：数组中重复的数字

题目：在一个长度为n的数组里的所有数字都在0到n-1的范围内。 数组中某些数字是重复的，但不知道有几个数字是重复的。也不知道每个数字重复几次。请找出数组中任意一个重复的数字。 例如，如果输入长度为7的数组[2,3,1,0,2,5,3]，那么对应的输出是第一个重复的数字2。将每个元素的出现的次数写到文件count.txt中
"""

list_num = [2,3,1,0,2,5,3,5,0,0]
dict_count = {} #用于存储元素出现的次数
for item in list_num:
 if item not in dict_count:
 dict_count[item] =1
 else:
 dict_count[item] = dict_count[item]+1
with open("count.txt","wt") as p1:
 for key,value in dict_count.items():
 p1.write("{} 次数{} \n".format(str(key),str(value)))
```

## 2.打开模式

Python 通过 `open()` 函数打开一个文件,并返回一个操作这个文件的变量,  
语法形式如下:

`<变量名> = open(<文件路径及文件名>, <打开模式>)`

| 打开模式 | 含义                                                    |
|------|-------------------------------------------------------|
| 'r'  | 只读模式,如果文件不存在,返回异常 <code>FileNotFoundError</code> ,默认值 |
| 'w'  | 覆盖写模式,文件不存在则创建,存在则完全覆盖源文件                             |
| 'x'  | 创建写模式,文件不存在则创建,存在则返回异常 <code>FileExistsError</code>   |
| 'a'  | 追加写模式,文件不存在则创建,存在则在原文件最后追加内容                          |
| 'b'  | 二进制文件模式                                               |
| 't'  | 文本文件模式,默认值                                            |
| '+'  | 与 <code>r/w/x/a</code> 一同使用,在原功能基础上增加同时读写功能           |

## (二) 读取

`readall()` 读取全文,以字符串表示

`>read([size])`

>功能: 来直接读取文件中字符。

>参数: 如果没有给定 `size` 参数 (默认值为-1) 或者 `size` 值为负,文件将被读取直至末尾,给定 `size` 最多读取给定数目个字符 (字节)。

>返回值: 返回读取到的内容

>

>\* 注意: 文件过大时候不建议直接读取到文件结尾,读到文件结尾会返回空字符串。

`>readline([size])`

>功能: 用来读取文件中一行

>参数: 如果没有给定 `size` 参数 (默认值为-1) 或者 `size` 值为负,表示读取一行,给定 `size` 表示最多读取指定的字符 (字节)。

>返回值: 返回读取到的内容

`>readlines([sizeint])`

>功能: 读取文件中的每一行作为列表中的一项

>参数: 如果没有给定 `size` 参数 (默认值为-1) 或者 `size` 值为负,文件将被读取直至末尾,给定 `size` 表示读取到 `size` 字符所在行为止。

>返回值: 返回读取到的内容列表

>文件对象本身也是一个可迭代对象,在 `for` 循环中可以迭代文件的每一行。

```
with open("jpg.jpg","rb") as f:
```

```
 for line in f:
```

```
 print(line)
```

```
#通用的复制器----使用b模式
def copy(ur1,ur12):
 """
 ur1 :被复制的文件的路径
 ur12:新文件
 """
 with open(ur1,"rb") as f1,open(ur12,"wb") as f2:
 for item in f1: #读出每行
 f2.write(item) #写进每行

copy("4010173121.jpg","js18150322.jpg")
```

### (三) 写

write(string)

功能: 把文本数据或二进制数据块的字符串写入到文件中去

参数: 要写入的内容

如果需要换行要自己在写入内容中添加\n

writelines(str\_list)

功能: 接受一个字符串列表作为参数, 将它们写入文件。

参数: 要写入的内容列表

4.

```
打开文件
fo = open("test.txt", "w")
print ("文件名为: ", fo.name)
seq = ["菜鸟教程 1\n", "菜鸟教程 2"]
fo.writelines(seq)

关闭文件
fo.close()
```

### 文本修改

```
#方法1, 占用额外的内存, 造成内存浪费
with open("c.txt",mode="rt",encoding="utf-8")as f:
 res = f.read()
 data = res.replace("<旧字符>","<新字符>")

with open("c.txt",mode="wt",encoding="utf-8") as f1:
 f1.write(data)

#方法二 占用额外的硬盘空间
import os
with open("c.txt",mode="rt",encoding="utf-8") as f,\
with open("c.txt.swap",mode="rt",encoding="utf-8") as f1:
 for item in f:
 f1.write(item.replace("", ""))
```

```
os.remove("c.txt")
os.rename("c.txt.swap", "c.txt")
```

## (四) 其他操作

### 1. 刷新缓冲区

缓冲:系统自动的在内存中为每一个正在使用的文件开辟一个缓冲区,从内存向磁盘输出数据必须先送到内存缓冲区,再由缓冲区送到磁盘中去。从磁盘中读数据,则一次从磁盘文件将一批数据读入到内存缓冲区中,然后再从缓冲区将数据送到程序的数据区。

刷新缓冲区条件:

1. 缓冲区被写满
2. 程序执行结束或者文件对象被关闭
3. 行缓冲遇到换行
4. 程序中调用flush()函数

```
f = open('a.py', 'w', 1) # 行缓冲
f = open('a.py', 'w') # 默认缓冲

while True:
 data = input(">>")
 if not data:
 break
 f.write(data + '\n')
 f.flush() # 刷新缓冲区

f.close()
```

flush()

该函数调用后会进行一次磁盘交互,将缓冲区中的内容写入到磁盘。

### 2. 文件偏移量

#### 1. 定义

打开一个文件进行操作时系统会自动生成一个记录,记录中描述了对文件的一系列操作。其中包括每次操作到的文件位置。文件的读写操作都是从这个位置开始进行的。

#### 2. 基本操作

tell()

功能: 获取文件偏移量大小

seek(offset[, whence])

功能: 移动文件偏移量位置

参数: offset 代表相对于某个位置移动的字节数。负数表示向前移动,正数表示向后移动。

whence是基准位置的默认值为0,代表从文件开头算起,1代表从当前位置算起,2代表从文件末尾算起。

- 必须以二进制方式打开文件时基准位置才能是1或者2

```
以r,w打开文件偏移量在开头，以a打开文件偏移量在结尾
f = open("mm.jpg", 'rb+')
print(f.tell())

f.write("Hello world")
#
print(f.tell())

以开头为基准向后移动5个字符
f.seek(1024, 0)

f.write('你好'.encode())
data = f.read()
print(data)

f.close()
```

### 3.文件描述符

#### 1. 定义

系统中每一个IO操作都会分配一个整数作为编号，该整数即这个IO操作的文件描述符。

#### 2. 获取文件描述符

`fileno()`

通过IO对象获取对应的文件描述符

## 八、库

### (一) 基础知识

#### 1.pip工具安装

```
pip工具安装
pip install <包名> 安装
pip uninstall <> 卸载
pip list 查询所有库
pip show <> 详细查询
pip download <> 下载
pip search <> 查询关键字
pip install (安装包名称) -i http://pypi.douban.com/simple/ --trusted-host
pypi.douban.com
阿里云 http://mirrors.aliyun.com/pypi/simple/
中国科技大学 https://pypi.mirrors.ustc.edu.cn/simple/
豆瓣(douban) http://pypi.douban.com/simple/
清华大学 https://pypi.tuna.tsinghua.edu.cn/simple/
中国科学技术大学 http://pypi.mirrors.ustc.edu.cn/simple/
```

## 2.模块变量

### (1) python程序结构

文件夹（项目根目录） -- 包 - 模块 - 类 - 函数 - 语句

包：将模块以文件夹的形式进行分组管理

`__all__`变量：定义可导出成员（当前模块），仅对`from <> import *`语句有效；

`__all__ = [ "", "" ]` #写在包的 `__init__.py` 文件中

`__doc__`变量：文档字符串（查看注释）；

`__file__`变量：模块对应的文件路径名；

`__name__`变量：模块自身名字，可以判断是否为主模块。

if `__name__ == "main"`: 当前模块作为主模块（第一个运行的模块）运行时，**name**绑定

作用1：

测试代码，只有从当前模块运行才会执行测试代码，只有从当前模块运行才会执行

作用2：

限制只能从当前模块才执行。限制只能从当前模块才执行。

## 3.模块

包含一系列数据、函数、类的文件，以.py结尾

作用：使结构清晰

调用模块（系统或第三方）

```
import <>
from <模块> import <具体模块> or *
import <模块名> as <别名>
#自定义模块在调用时，建议将项目主文件路径加入path中
#在启动文件中加入下列代码
3.5以下的版本
import sys #好处：项目内的可以跨文件夹调用
__file__ #显示当前文件的绝对路径
os.path.dirname(__file__) #__file__ 的父亲文件的路径
re=os.path.dirname(os.path.dirname(__file__)) #项目主文件夹
sys.path.append(re)
```

3.5以后的版本

```
from pathlib import Path
Path(__file__).parent.parent #项目主文件夹
res =Path('/a/a') / "d/e.txt" #拼接
res.resolve() #规范路径
```

## 4.包

定义

将模块以文件夹的形式进行分组管理

作用

使结构清晰

调用

```
from <包.模块> import <成员>
from <包.包.模块> import <成员>
import.<包>.<模块>
```

## (二) 系统操作库

### 1.os库

| 方法                                     | 说明                                            |
|----------------------------------------|-----------------------------------------------|
| os.getcwd()                            | 获取当前工作目录，即当前python脚本工作的目录路径                   |
| os.chdir ( "dirname")                  | 改变当前脚本工作目录;相当于shell下cd                        |
| os.curdir                              | 返回当前目录:"-")                                   |
| os.pardir                              | 获取当前目录的父目录字符串名: (..-")                        |
| os.makedirs ( 'dirname1/dirname2<br>") | 可生成多层递归目录                                     |
| os.removedirs ( 'dirname1')            | 若目录为空，则删除，并递归到上一级目录，如若也为空，则删除，依此类推            |
| os.mkdir ( 'dirname " )                | 生成单级目录;相当于shel1中mkdir dirname                 |
| os . rmdir ( 'dirname " )              | 删除单级空目录，若目录不为空则无法删除，报错;相当于shell中rmdir dirname |
| os.listdir ( "dirname ' )              | 列出指定目录下的所有文件和子目录，包括隐藏文件。并以列表方式打印              |
| os. remove()                           | 删除一个文件                                        |
| os.rename ( "oldname",<br>"newname")   | 重命名文件/目录                                      |
| os.stat( 'path/filename")              | 获取文件/目录信息                                     |
| os.sep                                 | 输出操作系统特定的路径分隔符,win下为"\1",Linux下为"/"           |
| os. linesep                            | 输出当前平台使用的行终止符，win下为"\r",Linux下为"\n"           |
| os.pathsep                             | 输出用于分割文件路径的字符串win下为; , Linux下为:               |
| os.name                                | 输出字符串指示当前使用平台。win->'nt'; Linux->'posix'       |
| os.system ( "bash command")            | 运行shell命令,直接显示                                |
| os.environ                             | 获取系统环境变量                                      |
| os.path . abspath (path)               | 返回path规范化的绝对路径 __ file __ 当前文件路径              |
| os.path.split (path)                   | 将path分割成目录和文件名二元组返回                           |



| 方法                                   | 说明                                                              |
|--------------------------------------|-----------------------------------------------------------------|
| os.path.dirname (path)               | 返回path的目录。其实就是os.path.split(path)的第一个元素 #返回上一级目录                |
| os.path.basename (path)              | 返回path最后的文件名。如何path以/或\结尾，那么就会返回空值。即os.path. split (path)的第二个元素 |
| os.path.exists (path)                | 如果path存在，返回True;如果path不存在，返回False                               |
| os.path.isabs (path)                 | 如果path是绝对路径,返回True                                              |
| os.path.isfile (path)                | 如果path是一个存在的文件，返回True。否则返回False.                                |
| os.path.isdir (path)                 | 如果path是一个存在的目录，则返回True。否则返回False path.isdir (r" ")              |
| os.path.join(path1[,path2[, .....]]) | 将多个路径组合后返回，第一个绝对路径之前的参数将被忽略                                     |
| os.path.getatime (path)              | 返回path所指向的文件或者目录的最后存取时间                                         |
| os.path.getmtime (path)              | 返回path所指向的文件或者目录的最后修改时间                                         |
| os.path.getsize (path)               | 获取文件大小                                                          |
|                                      |                                                                 |

## 2.sys库

| 方法              | 说明                               |
|-----------------|----------------------------------|
|                 |                                  |
| <b>sys.argv</b> | 命令行参数List，获取解释器后的参数，第一个元素是程序本身路径 |
| sys.exit (n)    | 退出程序,正常退出时exit (0)               |
| sys.version     | 获取Python解释程序的版本信息                |
| sys.maxint      | 最大的Int值                          |
| <b>sys.path</b> | 返回模块的搜索路径，初始化时使用PYTHONPATH环境变量的值 |
| sys.platform    | 返回操作系统平台名称                       |
|                 |                                  |

```
#通用的复制器----使用b模式
import sys
src1 = sys.argv[1] #复制的文件
src2 = sys.argv[2] #目标
def copy(ur1,ur12):
 """
 ur1 :被复制的文件的路径
 ur12:新文件
 """
```

```
with open(url,"rb") as f1,open(url2,"wb") as f2:
 for item in f1: #读出每行
 f2.write(item) #写进每行
```

shell 命令

python3 <文件名> copy <文件名> <新文件>

#打印进度条

```
import time
import random
recv_size = 0 #已下载的数据大小 Byte
total_size = 456894 #下载总量 Byte

def progress(recv_size,total_size):
 time.sleep(1)
 speed = random.randint(1024,2048)
 if total_size-recv_size>speed:
 recv_size += speed
 print_schedule(recv_size,total_size)
 else:
 recv_size += total_size-recv_size
 print_schedule(recv_size,total_size)

def print_schedule(recv_size,total_size): #打印进度条
 temp = recv_size/total_size
 print("\r{: <50}{}".format(int(temp*50)*"█",int(temp*100)), end="") #
```

## (三) 时间库

### 1.time库

| 方法                                      | 说明                             |
|-----------------------------------------|--------------------------------|
| .time()                                 | 获取当前时间戳（从1970年1月1日到现在的秒）可以算数运算 |
| .localtime( [<时间戳>] )                   | 时间元组,为空则获取当前时间                 |
| .mktime(<时间元组>)                         | 将时间元组=>时间戳                     |
| .strftime(<%Y-%m-%d %H:%M:%S>[,<时间元组>]) | 时间元组=>字符串，可以省略                 |
| .strptime("时间字符串",format)               | 字符串 =>时间元组                     |
| .sleep(s)                               | 程序停顿s秒                         |
| .asctime()                              | 作用与strftime相同                  |

%a      Locale's abbreviated weekday name.

%A Locale's full weekday name .  
 %b Locale's abbreviated month name.  
 %B Locale's full month name .  
 %c Locale's appropriate date and time representation.  
 %d Day of the month as a decimal number [01,31].  
 %H Hour (24-hour clock) as a decimal number [00,23].  
 %I Hour (12-hour clock) as a decimal number [01,12].  
 %j Day of the year as a decimal number [001,366].  
 %m Month as a decimal number [01,12].  
 %M Minute as a decimal number [00,59].  
 %p Locale's equivalent of either AM or PM.(1)  
 %S Second as a decimal number [00,61].(2)  
 %U week number of the year (Sunday as the first day of the week) as a decimal number[00,53]. All days in a new year preceding the first Sunday are considered to be in week 0.(3)  
 %w weekday as a decimal number [0(Sunday) ,6].  
 %W week number of the year (Monday as the first day of the week) as a decimal number[00,53]. All days in a new year preceding the first Monday are considered to be in week 0.(3)  
 %x Locale's appropriate date representation.  
 %X Locale's appropriate time representation.  
 %y year without century as a decimal number [00,99].  
 %Y year with century as a decimal number.  
 %z Time zone offset indicating a positive or negative time difference from UTC/GMT of the form +HHMM or -HHMM, where H represents decimal hour digits and M represents decimal minute digits [-23:59,+23:59].  
 %Z Time zone name (no characters if no time zone exists).



## 2.datetime模块

| 方法                                               | 说明                                               |
|--------------------------------------------------|--------------------------------------------------|
| <code>.datetime.now()</code>                     | 获取当前日期时间, 可以算数运算, 需要借用 <code>.timedelta()</code> |
| <code>.timedelta(year, month, day[, ...])</code> |                                                  |
|                                                  |                                                  |

```
import datetime
print(datetime.datetime.now()+datetime.timedelta(day=-3))
```

## (四) jieba库

| 方法                                    | 说明         |
|---------------------------------------|------------|
| <code>.lcut(s)</code>                 | 精确模糊       |
| <code>.lcut(s, cut_all = True)</code> | 全模式        |
| <code>add_word(w)</code>              | 向分词字典增加新词w |

## (五) 数学库

### 1.random库

| 方法                           | 说明                  |
|------------------------------|---------------------|
| seed()                       |                     |
| random()                     | 产生【0,1) 之间的小数、      |
| randint(a,b)                 | 产生【a,b]之间的整数        |
| getrandbits(k)               |                     |
| uniform(a,b)                 | (a,b)之间的小数          |
| choice(seq)                  | 随机返回seq容器里的内容       |
| randrange(start,stop[,step]) | 产生【start,stop)之间的整数 |
| shuffle(seq)                 | 打乱seq容器的顺序          |
| sample(pop,k)                | 随机返回pop容器里的k个内容     |

```
#验证码
import random
def get_verification_code(n):
 lists= []
 for i in range(n):
 randint = random.randint(0,10):
 if randint in (list(range(0,4))):
 lists.append(str(random.randint(0,9)))
 elif randint in (list(range(4,7))):
 lists.append(chr(random.randint(65,90)))
 elif randint in (list(range(7,10))):
 lists.append(chr(random.randint(97,122)))
 return "".join(lists)
```

## (六) 绘图库

### 1.turtle库

turtle(海龟)库是turtle绘图体系python的实现。

- turtle库运行原理

原理：有一只海龟通过从**程序控制**，**自由改变颜色**，**方向宽度**在**窗体正中心**游走，走过的痕迹可以绘制成图形。

**Turtle** 库是 Python 的一个标准库，主要用于图像的绘制。想象您用一组组函数驾驭一只小小的乌龟，在无垠的沙滩（画布）上昂首阔步，纵横驰骋，画出一个个令人惊艳的图形，甭提多有成就感。

- 画布

**画布** (canvas) , 也就是让海龟“挥毫泼墨”用于绘图的区域, 单位为像素。创作之前您可以设置需要的大小和背景。turtle模块中的**x轴正方向指向右侧, y轴正方向指向上方。坐标原点位于画布的中心**

设置画布的大小和背景颜色

```
turtle.screensize(canvwidth=None, canvheight=None, bg=None)
```

- canvwidth: 画布的宽度
- canvheight: 画布的高度
- bg: 背景颜色
- 当宽度或者高度为整数时表示的是像素; 小数时, 表示占据电脑屏幕的比例。当高度或者宽度超过窗口大小时, 会出现滚动条。
- 若不设置值, 默认参数为 (400,300, None)

如果要设定画布在屏幕中的初始位置, 则需要使用下列代码:

```
turtle.setup(width,height,startx,starty)
```

 turtle画布

参数 width、height 为画布的宽和高, 输入为整数时, 表示像素; 为小数时, 则表示占据电脑屏幕的比例, startx, starty 分别代表画布距离屏幕左、上边缘的像素距离, 空白表示画布位于屏幕中心。

参数 canvwidth 和 canvheight 分别为画布的宽和高, bg 为背景颜色。空白示返回默认大小(400, 300)。

- 画笔 (海龟)

在画布上, 默认有一个以画布中心为原点的坐标轴, 其上为一只面朝x轴方向的小乌龟。turtle绘图中, 就是根据海龟的位置方向等定义画笔的状态。

画笔属性: 颜色、画线的宽度、移动速度等。

turtle.pensize(): 设置画笔的宽度, 数字越大, 画笔越粗;

turtle.pencolor(): 没有参数传入, 返回当前画笔颜色, 传入参数设置画笔颜色, 可以是字符串如"green", "red",也可以是rgb3元组。注意: 当是rgb时, r、g、b取值为0.0~1.0之间。

```
turtle.pencolor(1.0, 0.0, 0.0)
#或者tup1=(1.0, 0.0, 0.0)
#turtle.pencolor(tup1) #当然列表等其他可迭代序列也可以。同样也可以使用
turtle.pencolor((tup1))

turtle.pencolor("red")
```

版权声明: 本文为CSDN博主「frist word」的原创文章, 遵循CC 4.0 BY-SA版权协议, 转载请附上原文出处链接及本声明。

原文链接: [https://blog.csdn.net/qq\\_51315420/article/details/121492391](https://blog.csdn.net/qq_51315420/article/details/121492391)

| 类型       | 方法                                 | 说明                                                      |
|----------|------------------------------------|---------------------------------------------------------|
| 窗体<br>函数 | .setup(width,height,startx,starty) |                                                         |
|          | .done()                            | 停止画笔绘制，窗体不关闭                                            |
| 画笔<br>属性 |                                    |                                                         |
|          | pensize(width)                     | 画笔线条粗细                                                  |
|          | pencolor(colorstring)              | 画笔的颜色                                                   |
|          | .speed(speed)                      | 设置画笔移动速度，画笔绘制的速度范围[0,10]整数，数字越大越快，移动速度越快。               |
|          | .fillcolor(colorstring)            | 填充颜色                                                    |
|          | color((color1, color2)             | 设置画笔颜色和填充颜色                                             |
|          | begin_fill()                       | 填充颜色开始的函数                                               |
|          | end_fill()                         | 填充颜色结束的函数                                               |
|          | .hideturtle()                      | 隐藏画笔的turtle形状                                           |
|          | .showturtle()                      | 显示画笔的turtle形状                                           |
| 画笔<br>运动 |                                    |                                                         |
|          | penup()                            | 提起画笔                                                    |
|          | pendown()                          | 放下画笔                                                    |
|          | forward(distance)                  | 画笔向前运动                                                  |
|          | backward(distance)                 | 画笔向后运动                                                  |
|          | right(angle)                       | 向右旋转angle角度                                             |
|          | left(angle)                        | 向左旋转angle角度                                             |
|          | goto(x,y)                          | 移动到绝对坐标 (x,y)处                                          |
|          | setx( )                            | 将当前x轴移动到指定位置                                            |
|          | sety( )                            | 将当前y轴移动到指定位置                                            |
|          | seth(to_angle)                     | 旋转多少度                                                   |
|          | circle(radiua,e/steps)             | 绘制一个半径为r，角度e的圆或弧形，或steps=3正3边行,半径为正(负)，表示圆心在画笔的左边(右边)画圆 |

| 类型     | 方法                                                          | 说明                                   |
|--------|-------------------------------------------------------------|--------------------------------------|
| 全局控制命令 |                                                             |                                      |
|        | .done()                                                     | 使绘图窗口不会自动消失                          |
|        | .clear()                                                    | 清空turtle窗口，但是turtle的位置和状态不会改变        |
|        | .reset()                                                    | 清空窗口，重置turtle状态为起始状态                 |
|        | .undo()                                                     | 撤销上一个turtle动作                        |
|        | turtle.write(s [,font=("font-name",font_size,"font_type")]) | #写文本，s为文本内容，font是字体的参数，分别为字体名称，大小和类型 |

## (七) 文件库

后期加入

## (八) shutil模块

高级的文件、文件夹、压缩包处理模块

shutil.copyfileobj(src, dst[, length]) 将文件内容拷贝到另一个文件中

```
import shutil
shutil.copyfileobj(open('old.xml','r'),open('new.xml','w'))
```

shutil.copyfile(src, dst) 拷贝文件

```
shutil.copyfile('f1.log','f2.log')#目标文件无需存在
```

shutil.copymode(src, dst) 仅拷贝权限。内容、组、用户均不变

```
shutil.copymode('f1.log','f2.log')#目标文件必须存在
```

shutil.copystat(src, dst) 仅拷贝状态的信息，包括: mode bits, atime, mtime, flags

```
shutil.copystat('f1.log','f2.log')#目标文件必须存在
```

shutil.copy(sre, dst)拷贝文件和权限

```
import shutil
shutil.copy('t1.log','r2.log')
```

shutil.copy2(src, dst) 拷贝文件和状态信息

```
shutil.copy2('f1.log','f2.log')
```

shutil.ignore\_patterns(patterns)

shutil.copytree(src, dst, symlinks=False, ignore=None)递归的去拷贝文件夹

```
import shutil
shutil.copytree('folder1', 'folder2 ', ignore=shutil.ignore_patterns ('*.pyc', 'tmp* '))#目标目录不能存在，注意对folder2目录父级目录要有可写权限， ignore的意思是排除田拷贝软连接
```

shutil.rmtree(path[, ignore\_errors[, onerror]]) 递归的去删除文件

```
!import shutil2
shutil.rmtree ('folder1')
```

shutil.move(src, dst)

递归的去移动文件，它类似mv命令，其实就是重命名。

```
import shutil
shutil.move ('folder1', 'folder3 ')
```

shutil.make\_archive(base\_name, format,...)创建压缩包并返回文件路径，例如:zip、tar创建压缩包并返回文件路径，例如:zip、tar

. base\_name:压缩包的文件名，也可以是压缩包的路径。只是文件名时，则保存至当前目录，否则保存至

指定路径，

如data\_bak

=>保存至当前路径

如:/tmp/data\_bak =>保存至/tmp/

. format:压缩包种类,"zip","tar", "bztar", "gztar". root\_dir:要压缩的文件夹路径(默认当前目录). owner:用户，默认当前用户

. group:组，默认当前组

. logger:用于记录日志,通常是logging.Logger对象

```
#将/ data下的文件打包放置当前程序目录import shutil
ret = shutil.make_archive ("data_bak", 'gztar', root_dir='/data ')
#将/data下的文件打包放置/ tmp/目录
import shutil
ret = shutil.make_archive (" /tmp/data_bak",'gztar', root_dir='/data ')
```

## (九) json模块&pickle模块

### 1.序列化

序列化指的是把内存的数据类型转换成一个特定的格式的内容

内存中的数据类型---->序列化---->特定的格式(json格式或者pickle格式)

内存中的数据类型<---反序列化<---特定的格式(json格式或者pickle格式) (反序列化)

作用:

该格式的内容可用于存储(存档) pickle 只有python独有

传输给其他平台使用(跨平台) json 被所有语言识别



## 2.json

如果我们要在不同的编程语言之间传递对象，就必须把对象序列化为标准格式，比如XML，但更好化为JSON，因为JSON表示出来就是一个字符串，可以被所有语言读取，也可以方便地存储到磁盘传输。JSON不仅是标准格式，并且比XML更快，而且可以直接在Web页面中读取，非常方便。JSON表示的对象就是标准的JavaScript语言的对象，JSON和Python内置的数据类型对应如下：

| JSON类型     | Python类型   |
|------------|------------|
| {}         | dict       |
| []         | list       |
| "string"   | str        |
| 1234.56    | int float  |
| true false | True False |
| null       | None       |

### 方法

| 方法                 | 说明               |
|--------------------|------------------|
| dumps(<>)          | 转化为序列(字符串)       |
| .loads(<序列>)       | 序列 ->代码(与eval相似) |
| dump( <内容>,<文件变量>) | 将序列化的内容保存到文件内    |

```
import json
res = json.dumps(True) #"true"
res = json.dumps([1,2]) #"[1,2]"
with open("text.txt","wt") as p:
 p.write(res)

with open ("text.json","wt") as f:
 json.dump(res,[5,4])
```

注：ujson模块与json的作用和用法相同，但比json 效率高

## 3.pickle模块

### 方法

| 方法                 | 说明               |
|--------------------|------------------|
| dumps(<>)          | 转化为序列(字符串)       |
| .loads(<序列>)       | 序列 ->代码(与eval相似) |
| dump( <内容>,<文件变量>) | 将序列化的内容保存到文件内    |

```

import pickle
res = pickle.dumps(True) #"true"
res = pickle.dumps([1,2]) #"[1,2]"
with open("text.txt","wt") as p:
 p.write(res)

with open ("text.json","wt") as f:
 pickle.dump(res,["5",4])

```

## (十) configparser模式

#配置文件

```

#语法
#定义
import configparser
[section1]
k1 = v1
k2 : v2
user=egonage=18
is_admin=truesalary=31
[section2]
k1 = v1

#调用
import configparser
#将配置文件加载到内存
config =configparser.ConfigParser()
config.read("test.ini")
#获取配置内容
config.sections() #["section1","section2"]
#获取某一section下的所有options
config.options("section1") #["k1","k2","user","is_admin"]
#获取items
config.items("sections1") #[('k1', 'v1'), ('k2','v2'),...]

config.get("sections1","user") #"18" str

config.getint("sections1","user") #18 int

config.getboolean("sections1","user") #True

config.getfloat("sections1","user") #18.0

```

## (十一) hashlib模块

### 1.概念

1、什么叫hash:hash是一种算法 (3.x里代替了md5模块和sha模块, 主要提供SHA1, SHA224, SHA512,MD5算法), 该算法接受传入的内容, 经过运算得到一串hash值

#2、hash值的特点是:

#2.1只要传入的内容一样, 得到的hash值必然一样====>要用明文传输密码文件完整性校验

#2.2不能由hash值返解成内容=====》把密码做成hash值, 不应该在网络传输明文密码

#2.3只要使用的hash算法不变, 无论校验的内容有多大, 得到的hash值长度是固定的

### 2.语法

```
import hashlib

hash = hashlib.md5() #哈希生成器
hash.update("hello".encode("utf-8")) #输入需要转化的字符
hash.hexdigest() #输出结果
```

### 3.密码加盐--增加复杂度

```
#模拟撞库
hashpaa = " "
password_list = [
 "123",
 "1234"
] #撞库的密码
import hashlib
for item in password_list:
 hash = hashlib.md5()
 hash.update(item.encode("utf-8"))
 if hash.hexdigest() == hashpaa:
 print(item,"找到了")
else:
 print("没找到")

#密码加盐--实质是在密码中增加额外部分, 增加复杂度, 增加撞库的成本
import hashlib
hash.update("墨非墨".encode("utf-8"))
hash.update("密码部分".encode("utf-8"))
hash.hexdigest()
```

## (十二) subprocess模块

```
import subprocess
#执行“ ”里shell命令 查看文件夹的内容
obj=subprocess.Popen("ls /root",shell=True,
 stdout=subprocess.PIPE,#将正确的结果传到 stdout
 stderr=subprocess.PIPE)#将错误的结果传到 stderr
obj.stdout.read() #查看正确结果
obj.stderr.read().decode("utf-8")#查看错误结果,以utf8编码格式
```

## (十二) 日志模块

```
#settings.py
import os
import logging
#一: 日志配置
logging.basicConfig(

 """

#1、日志输出位置: 1、终端 2、文件
#filename='access.log',#不指定, 默认打印到终端
#2、日志格式
#format='%(asctime)s - %(name)s - %(levelname)s -&(module)s: %(message)s', #时间
名字 等级 哪一行

1、定义三种日志输出格式,日志中可能用到的格式化串如下
%(name)s Logger的名字
%(lineno)s 数字形式的日志级别
%(levelname)s 文本形式的日志级别
%(pathname)s 调用日志输出函数的模块的完整路径名, 可能没有
%(filename)s 调用日志输出函数的模块的文件名
%(module)s 调用日志输出函数的模块名
%(funcName)s 调用日志输出函数的函数名
%(lineno)d 调用日志输出函数的语句所在的代码行
%(created)f 当前时间, 用UNIX标准的表示时间的浮点数表示
%(relativeCreated)d 输出日志信息时的, 自Logger创建以来的毫秒数
%(asctime)s 字符串形式的当前时间。默认格式是“2003-07-08 16:49:45,896”。逗号后面的是毫秒
%(thread)d线程ID。可能没有
%(threadName)s 线程名。可能没有
(process)d进程ID。可能没有
%(message)s用户输出的消息

2、强调: 其中的%(name)s为getlogger时指定的名字
#3、时间格式 自定义
datefmt='%Y-%m-%d%H:%M:%S %p',

#4、日志级别
critical ->50
error =>40
warning =>30
info ->20
debug -=>10
level=10,
```

```

"""

#1.输出日志的模板
standard_format = '%(asctime)s - %(threadName)s:%(thread)d - 日志名:%(name)s - %(filename)s:%(lineno)d - \n %(levelname)s - %(message)s'

simple_format= '%(levelname)s][%(asctime)s][%(filename)s:%(lineno)d]%(message)s'

test_format = '%(asctime)s%(message)s'

#2.配置字典
LOGGING_DIC = {
 "version" : 1,

 'disable_existing_loggers': False,

 #展示日志格式
 'formatters': {
 'standard': {
 'format': standard_format #调用日志模板
 },
 'simple': {
 'format': simple_format
 },
 "test": {
 'format': test_format
 },
 },

 'filters': {},

 #控制日志输出位置
 'handlers': {
 #打印到终端的日志
 "console": {
 'level': 'DEBUG', #日志级别
 'class': 'logging.StreamHandler', #打印到屏幕
 'formatter': 'simple' #展示日志格式},

 #打印到文件的日志收集infQ及以上的日志
 "default": {'level': 'DEBUG', #日志级别
 'class': 'logging.handlers.RotatingFileHandler', #保存到文件, 并开启轮转功能
 'formatter': 'standard',
 #可以定制日志文件路径
 'BASE_DIR': os.path.dirname(os.path.abspath(__file__)),
 'LOG_PATH': os.path.join(BASE_DIR, 'a1.log '),
 'filename': 'a1.log', #日志文件
 'maxBytes': 1024*1024*5, #日志大小5M
 'backupCount': 5, #最多轮转5次
 'encoding': 'utf-8', #日志文件的编码
 },
 },
}

```

```

 'other' : {
 'level': 'DEBUG' ,
 'class' : 'logging.FileHandler', #保存到文件
 'formatter' : 'test',
 'filename' : 'a2.log ',
 #os.path.join(os.path.dirname(os.path.dirname(__file__)), 'a2.log ')项目
 #文件
 'encoding' : 'utf-8' ,

 },
 },

 #产生不同级别日志,日志产生者
 'loggers' : {
 #logging.getLogger(__name__)拿的logger配置
 "<日志名> " : {
 'handlers' : ['default' , 'console'], #"<变量>"产生的日志丢给[]
 'level': 'DEBUG' , # loggers(第一层日志级别关限制)--->handlers(第二层日志
 #级别关限制) 可以输出aaa.debug("")级别高的 .info()
 'propagate' : False, #默认为True, 向上(更高level的logger)传递, 通常设False
 },
 "a":{
 'handlers' : ['other'],
 'level' : 'DEBUG' ,
 'propagate' : False,},
 },
 "b":{
 'handlers' : ['default'],
 'level' : 'DEBUG' ,
 'propagate' : False,},
 },
},
}

```

#二:输出日志

```

logging.debug('调试debug ') #往下, 等级越高
logging.info('消息info')
logging.warning("警告warn ")
logging.error("错误error")
logging.critical ('严重critical ')
"""

```

·注意下面的root是默认的日志名字

```

WARNING : root:警告warn
ERROR : root:错误error
CRITICAL: root:严重critical
"""

```

#使用日志的文件

```

import settings

```

```
from logging import config, getLogger

#导入日志配置
#1. 导入配置字典
config.dictConfig(settings.LOGGING_DIC)
#拿到日志的产生者
logger1=getLogger("a")
logger1.info("") #产生日志
logger2=getLogger("b")
logger2.info("") #产生日志
```

日志名：是用于区分日志业务  
日志轮转 #为了防止日志文件太大，所进行的分割  
日志记录者程序员运行过程中的关键信息

## (十三) 正则表达式(re)

### 1.简介

#### 定义

即文本的高级匹配模式，提供搜索，替换等功能。其本质是由一系列字符和特殊符号构成的字符串，这个字符串即正则表达式。

#### 原理

通过普通字符和有特定含义的字符，来组成字符串，用以描述一定的字符串规则，比如：重复，位置等，来表达某类特定的字符串，进而匹配。

### 2.元字符

| 字符  | 说明            |
|-----|---------------|
| .   | 匹配除换行符以外的任意字符 |
| \w  | 匹配字母或数字或下划线   |
| \s  | 匹配任意的空白符      |
| \d  | 匹配数字          |
| \n  | 匹配一个换行符       |
| \t  | 匹配一个制表符       |
| ^   | 匹配字符串的开始      |
| \$  | 匹配字符串的结尾      |
| \W  | 匹配非字母或数字或下划线  |
| \D  | 匹配非数字         |
| \S  | 匹配非空白符        |
| a b | 匹配字符a或字符b     |

| 字符     | 说明               |
|--------|------------------|
| ()     | 匹配括号内的表达式，也表示一个组 |
| [...]  | 匹配字符组中的字         |
| [^...] | 匹配除了字符组中字符的所有字符  |

#### (1)普通字符

- 匹配规则：每个普通字符匹配其对应的字符

```
email = "墨非墨：1433830053@qq.com"
import re
re.findall("\w+@\w+\.\cn") #1433830053@qq.com
```

- 注意事项：正则表达式在python中也可以匹配中文

#### (2)或关系

- 元字符：|
- 匹配规则：匹配 | 两侧任意的正则表达式即可

```
e.g.
In : re.findall('com|cn','www.baidu.com/www.tmooc.cn')
Out: ['com', 'cn']
```

#### (3)匹配单个字符

- 元字符：.
- 匹配规则：匹配除换行外的任意一个字符

```
import re
re.findall("贾.春","贾元春,贾迎春,贾探春,贾惜春")
#["贾元春", "贾迎春", "贾探春", "贾惜春"]
```

#### (4)匹配字符集

- 元字符：[字符集]
- 匹配规则：匹配字符集中的任意一个字符
- 表达形式：

[abc#!好] 表示 [] 中的任意一个字符  
 [0-9],[a-z],[A-Z] 表示区间内的任意一个字符  
 [\_#?0-9a-z] 混合书写，一般区间表达写在后面

```
import re
re.findall(["元迎探惜"],"贾元春,贾迎春,贾探春,贾惜春")
#["元", "迎", "探", "惜"]
re.findall([0-9],"今年2022年")
#["2", "0", "2", "2"]
```

#### (5)匹配字符集反集



- 元字符: [^字符集]
- 匹配规则: 匹配除了字符集以外的任意一个字符

```
e.g.
In : re.findall('[^0-9]','Use 007 port')
Out: ['U', 's', 'e', ' ', ' ', ' ', 'p', 'o', 'r', 't']
```

(6)匹配字符串开始位置

- 元字符: ^
- 匹配规则: 匹配目标字符串的开头位置

```
import re
re.findall("^林黛玉|薛宝钗", "贾元春,贾迎春,贾探春,贾惜春,林黛玉,薛宝钗")
#["林黛玉", "薛宝钗"]

re.findall('[^]+', "Port-9 Error #404# %@STD")
['Port-9', 'Error', '#404#', '%@STD']
```

(7) 匹配字符串的结束位置

- 元字符: \$
- 匹配规则: 匹配目标字符串的结尾位置

```
e.g.
In : re.findall('Jame$', "Hi, Jame")
Out: ['Jame']
```

- 规则技巧: ^ 和 \$必然出现在正则表达式的开头和结尾处。如果两者同时出现，则中间的部分必须匹配整个目标字符串的全部内容。

量词

| 字符    | 说明       |
|-------|----------|
| *     | 重复零次或更多次 |
| +     | 重复一次或更多次 |
| ?     | 重复零次或一次  |
| {n}   | 重复n次     |
| {n,}  | 重复n次或更多次 |
| {n,m} | 重复n到m次   |

(8)匹配字符重复

- 元字符: \*
- 匹配规则: 匹配前面的字符出现0次或多次

```
e.g.
In : re.findall('wo*', 'wooooo~w!')
Out: ['wooooo', 'w']
import re
re.findall("[0-9]*", "MFM13")
#["13"]
re.findall('[A-Z][a-z]*', "Hello world") #匹配一个大写字母+多个小写
```

- 元字符: +
- 匹配规则: 匹配前面的字符出现1次或多次

```
e.g.
In : re.findall('[A-Z][a-z]+', "Hello world")#匹配一个大写字母+至少1个小写
Out: ['Hello', 'world']
```

- 元字符: ?
- 匹配规则: 匹配前面的字符出现0次或1次

```
import re
re.findall(".春?", "贾元春,贾迎春,贾探春,贾惜春,林黛玉,薛宝钗")

>>> re.findall('-[0-9]*|[0-9]*', "167,-28,29,-8")
['167', '', '-28', '', '29', '', '-8', '']

>>> re.findall('-?[0-9]*', "167,-28,29,-8")
['167', '-28', '29', '-8', '']
```

- 元字符: {n}
- 匹配规则: 匹配前面的字符出现n次

```
e.g. 匹配手机号码
In : re.findall('1[0-9]{10}', "Jame:13886495728")
Out: ['13886495728']

re.findall('张.{2}', "张飞 张翼德")
['张飞 ', '张翼德']
```

- 元字符: {m,n}
- 匹配规则: 匹配前面的字符出现m-n次

```
e.g. 匹配qq号
In : re.findall('[1-9][0-9]{5,10}', "Baron:1259296994")
Out: ['1259296994']
```

(9)匹配任意（非）数字字符

- 元字符: \d \D
- 匹配规则: \d 匹配任意数字字符, \D 匹配任意非数字字符

e.g. 匹配端口  
In : re.findall('\d{1,5}', "mysql: 3306, http:80")  
Out: ['3306', '80']

(10)匹配任意（非）普通字符

- 元字符: \w \W
- 匹配规则: \w 匹配普通字符, \W 匹配非普通字符
- 说明: 普通字符指数字, 字母, 下划线, 汉字。

e.g.  
In : re.findall('\w+', "server\_port = 8888")  
Out: ['server\_port', '8888']

(11)匹配任意（非）空字符

- 元字符: \s \S
- 匹配规则: \s 匹配空字符, \S 匹配非空字符
- 说明: 空字符指 空格 \r \n \t \v \f 字符

e.g.  
In : re.findall('\w+\s+\w+', "hello world")  
Out: ['hello world']

(12)匹配开头结尾位置

- 元字符: \A \Z
- 匹配规则: \A 表示开头位置, \Z 表示结尾位置

(13)匹配（非）单词的边界位置

- 元字符: \b \B
- 匹配规则: \b 表示单词边界, \B 表示非单词边界
- 说明: 单词边界指数字字母(汉字)下划线与其他字符的交界位置。

e.g.  
In : re.findall(r'\bis\b', "This is a test.")  
Out: ['is']

| 类别   | 元字符                               |
|------|-----------------------------------|
| 匹配字符 | . [...] [^ ...] \d \D \w \W \s \S |
| 匹配重复 | * + ? {n} {m,n}                   |
| 匹配位置 | ^ \$ \A \Z \b \B                  |

| 类别 | 元字符                  |
|----|----------------------|
| 其他 | <code>[] () \</code> |

### 3.正则表达式的转义

1. 如果使用正则表达式匹配特殊字符则需要加 \ 表示转义。

特殊字符: `. * + ? ^ $ [] () { } | \`

e.g. 匹配特殊字符 `.` 时使用 `\.` 表示本身含义

```
In : re.findall('-?\d+\.?\d*', "123,-123,1.23,-1.23")
Out: ['123', '-123', '1.23', '-1.23']
```

1. 在编程语言中，常使用原生字符串书写正则表达式避免多重转义的麻烦。

e.g.

python字符串 `-->` 正则 `-->` 目标字符串  
`"\\$\\d+"` 解析为 `\\$\\d+` 匹配 `"$100"`

`"\\$\\d+"` 等同于 `r"\\$\\d+"`

```
>>> re.findall('-?[0-9]*\\.?[0-9]?',"167,-28,29,-8,-3.8")
['-28', '-8', '-3.8']
>>> re.findall('-?\\d+\\.?\\d+', "167,-28,29,-8,-3.8")
['-28', '-8', '-3.8']
```

```
re.findall(r'-?\\d+\\.?\\d+', "167,-28,29,-8,-3.8") #标准格式
['-28', '-8', '-3.8']
```

### 4.贪婪模式和非贪婪模式

1. 定义

贪婪模式: 默认情况下，匹配重复的元字符总是尽可能多的向后匹配内容。比如: `* + ? {m,n}`

非贪婪模式(懒惰模式): 让匹配重复的元字符尽可能少的向后匹配内容。

1. 贪婪模式转换为非贪婪模式

- 在匹配重复元字符后加 '?' 号即可

```
* : *?
+ : +?
? : ??
{m,n} : {m,n}?
```

e.g.

```
In : re.findall(r'\(.(+)\)', "(abcd)efgh(higk)") #贪婪模式
Out: "(abcd)efgh(higk)"
In : re.findall(r'\(.(+?)\)', "(abcd)efgh(higk)")
Out: ['(abcd)', '(higk)']
```

```
s = "[林黛玉],[贾府],[萍儿]"
re.findall(r'\[\w+\]',s)
```

1. 匹配一个.com邮箱格式字符串

```
str = "1433830053@qq.com"
re.findall(r'\w+@\w+.com$',str)
#['1433830053@qq.com']
```

2. 匹配一个密码8-12位数字字母下划线构成

```
re.findall(r'\w{8,12}',pass)
```

3. 匹配一个数字正数，负数，整数，小数，分数1/2，百分数45%

```
strs = "45 -45 4.5 20.45 1/2 45%"
re.findall(r'-?\d+\.?/?\d%?+',strs)
#['45', '-45', '4.5', '20.45', '1/2', '45%']
```

4. 匹配一段文字中以大写字母开头的单词，注意文字中可能有iPython(不算)H-base(算)  
单词可能有大写字母小写字母-」x

```
strs = "Hello H-base iPython"
re.findall(r'^[A-Z][_-a-zA-Z]*',strs)
```

## 5.正则表达式分组

### 1. 定义

在正则表达式中，以()建立正则表达式的内部分组，子组是正则表达式的一部分，可以作为内部整体操作对象。

### 1. 作用

- 可以被作为整体操作，改变元字符的操作对象

e.g. 改变 +号 重复的对象

```
In : re.search(r'(ab)+',"ababababab").group()
Out: 'ababababab'
```

e.g. 改变 |号 操作对象

```
In : re.search(r'(王|李)\w{1,3}',"王者荣耀").group()
Out: '王者荣耀'
```

- 可以通过编程语言某些接口获取匹配内容中，子组对应的内容部分

e.g. 获取url协议类型

```
re.search(r'(https|http|ftp|file):\/\/\S+', "https://www.baidu.com").group(1)
#https
```

### 1. 捕获组

可以给正则表达式的子组起一个名字，表达该子组的意义。这种有名称的子组即为捕获组。

格式：(P<name>pattern)

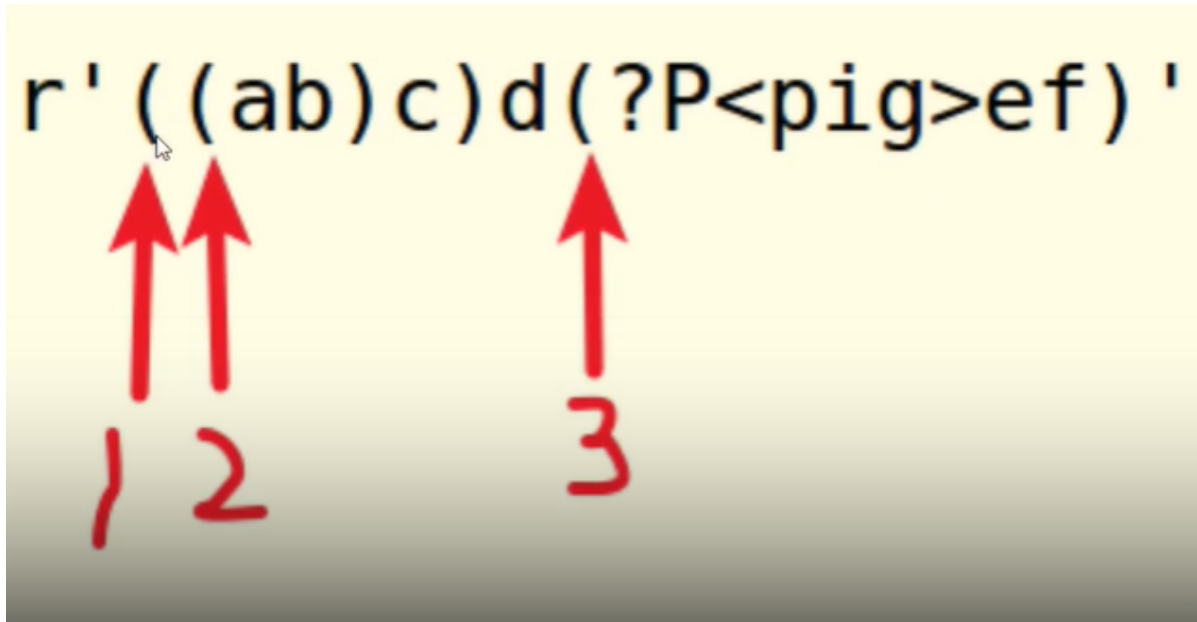
e.g. 给予组命名为 "pig"

```
In : re.search(r'(?P<pig>ab)+','ababababab').group('pig')
```

```
Out: 'ab'
```

#### 1. 注意事项

- 一个正则表达式中可以包含多个子组
- 子组可以嵌套，但是不要重叠或者嵌套结构复杂
- 子组序列号一般从外到内，从左到右计数



## 6.正则表达式匹配原则

1. 正确性,能够正确的匹配出目标字符串.
2. 排他性,除了目标字符串之外尽可能少的匹配其他内容.
3. 全面性,尽可能考虑到目标字符串的所有情况,不遗漏.

## 7.Python re模块使用

```
re = compile(pattern, flags = 0)
```

功能：生产正则表达式对象

参数：pattern 正则表达式

flags 功能标志位,扩展正则表达式的匹配

返回值：正则表达式对象

```
re.findall(pattern,string, flags = 0)
```

功能：根据正则表达式匹配目标字符串内容

参数：pattern 正则表达式

string 目标字符串

flags 功能标志位,扩展正则表达式的匹配

返回值：匹配到的内容列表,如果正则表达式有子组则只能获取到子组对应的内容

```
regex.findall(string,pos,endpos)
```

功能：根据正则表达式匹配目标字符串内容

参数：string 目标字符串

pos 截取目标字符串的开始匹配位置

endpos 截取目标字符串的结束匹配位置

返回值：匹配到的内容列表,如果正则表达式有子组则只能获取到子组对应的内容

---

```
re.split(pattern,string,flags = 0)
```

功能：使用正则表达式匹配内容,切割目标字符串

参数：pattern 正则表达式

string 目标字符串

flags 功能标志位,扩展正则表达式的匹配

返回值：切割后的内容列表

---

```
re.sub(pattern,replace,string,max,flags = 0)
```

功能：使用一个字符串替换正则表达式匹配到的内容

参数：pattern 正则表达式

replace 替换的字符串

string 目标字符串

max 最多替换几处,默认替换全部

flags 功能标志位,扩展正则表达式的匹配

返回值：替换后的字符串

---

```
re.subn(pattern,replace,string,max,flags = 0)
```

功能：使用一个字符串替换正则表达式匹配到的内容

参数：pattern 正则表达式

replace 替换的字符串

string 目标字符串

max 最多替换几处,默认替换全部

flags 功能标志位,扩展正则表达式的匹配

返回值：替换后的字符串和替换了几处

---

```
import re
```

```
目标字符串
```

```
s = "Alex:1994,Sunny:1999"
```

```
pattern = r"(\w+):(\d+)" # 正则表达式
```

```
re调用函数
```

```
l = re.findall(pattern,s)
```

```
print(l)
```

```
regex表达式对象调用函数
```

```
regex = re.compile(pattern)
```

```
l = regex.findall(s,) # 截取s[0:13]作为匹配目标
```

```
print(l)
```

```
按照正则匹配的内容,分割目标
```

```
l = re.split(r'[,:]',s)
print(l)

使用字符串替换匹配到的部分
s = re.subn(r':', '-', s, 1)
print(s)
```

---

```
re.finditer(pattern,string,flags = 0)
```

功能：根据正则表达式匹配目标字符串内容

参数： **pattern** 正则表达式  
      **string** 目标字符串  
      **flags** 功能标志位,扩展正则表达式的匹配

返回值：匹配结果的迭代器

---

```
re.fullmatch(pattern,string,flags=0)
```

功能：完全匹配某个目标字符串

参数： **pattern** 正则  
      **string** 目标字符串

返回值：匹配内容 **match object**

---

```
re.match(pattern,string,flags=0)
```

功能：匹配某个目标字符串开始位置

参数： **pattern** 正则  
      **string** 目标字符串

返回值：匹配内容 **match object**

---

```
re.search(pattern,string,flags=0)
```

功能：匹配目标字符串第一个符合内容

参数： **pattern** 正则  
      **string** 目标字符串

返回值：匹配内容 **match object**

---

## compile对象属性

- 【1】 **pattern** : 正则表达式
- 【2】 **groups** : 子组数量
- 【3】 **groupindex** : 捕获组名与组序号的字典

---

```
"""
regex1.py re模块 功能函数演示2
生成 match 对象
"""

import re
```



```

s = "今年是2019年,建国70周年"

pattern = r"\d+"
返回迭代对象
it = re.finditer(pattern,s)
for i in it:
 print(i.group()) # 获取match对象对应内容

完全匹配
obj = re.fullmatch(r'.+',s)
print(obj.group())

匹配开始位置
obj = re.match(r'\w+',s)
print(obj.group())

匹配第一处
obj = re.search(r'\d+',s)
print(obj.group())

```

## 8.match对象的属性方法

### 参考代码day13/regex2.py

#### 1. 属性变量

- pos 匹配的目标字符串开始位置
- endpos 匹配的目标字符串结束位置
- re 正则表达式
- string 目标字符串
- lastgroup 最后一组的名称
- lastindex 最后一组的序号

#### 1. 属性方法

- span() 获取匹配内容的起止位置
- start() 获取匹配内容的开始位置
- end() 获取匹配内容的结束位置
- groupdict() 获取捕获组字典，组名为键，对应内容为值
- groups() 获取子组对应内容
- group(n = 0)

功能：获取match对象匹配内容

参数：默认为0表示获取整个match对象内容，如果是序列号或者组名则表示获取对应子组内容

返回值：匹配字符串

```

"""
regex2.py
match对象属性实例

```

```

"""

import re

pattern = r"(ab)cd(?P<pig>ef)"
regex = re.compile(pattern)
obj = regex.search("abcdefghi",0,7) # match对象

属性变量
print(obj.pos) # 目标字符串开始位置
print(obj.endpos) # 目标字符串结束位置
print(obj.re) # 正则
print(obj.string) # 目标字符串
print(obj.lastgroup) # 最后一组组名
print(obj.lastindex) # 最后一组序号

print("=====")
属性方法
print(obj.span()) # 匹配到的内容在目标字符串中的位置
print(obj.start())
print(obj.end())
print(obj.groups()) # 子组内容对应的元组
print(obj.groupdict()) # 捕获组字典
print(obj.group()) # 获取match对象内容
print(obj.group('pig'))

```

## 9.flags参数扩展

1. 使用函数：re模块调用的匹配函数。如：re.compile,re.findall,re.search....
2. 作用：扩展丰富正则表达式的匹配功能
3. 常用flag

A == ASCII 元字符只能匹配ascii码

I == IGNORECASE 匹配忽略字母大小写

S == DOTALL 使 . 可以匹配换行

M == MULTILINE 使 ^ \$可以匹配每一行的开头结尾位置

1. 使用多个flag

方法：使用按位或连接

e.g. : flags = re.I | re.A

```

"""
flags.py 扩展功能标志
"""
import re

s = """Hello
北京"""

只能匹配ascii码
regex = re.compile(r'\w+',flags=re.A)

```

```

忽略字母大小写
regex = re.compile(r'[a-z]+', flags=re.I)

让 . 可以匹配\n
regex = re.compile(r'.+', flags = re.S)

^ $ 可以匹配每行的开始结束位置
regex = re.compile(r'Hello$', flags=re.M)

l = regex.findall(s)
print(l)

```

## 九、迭代器与生成器

### (一) 迭代器

#### 1. 可迭代对象 iterable

(1) 迭代--每一次对过程的重复，每一次迭代得到的结果会作为下一次迭代的初始值

(2) 可迭代对象--具有**iter**函数的对象

(3) 语法

```

"""--创建:.
 class可迭代对象名称:
 def __iter__(self):
 return 迭代器
--使用:.
 for 变量名 in 可迭代对象: #原理: 迭代器 = 可迭代对象.__iter__()
 语句.
"""

```

"""面试题:

#### 1. for循环的原理?

```

"""
#获取迭代器
it = list02.__iter__()
#循环获取下一个元素
while True:
 try:
 item = it.__next__()
 print(item)
 except StopIteration:
 break
 #遇到异常停止迭代
"""

```

#### 2. 可以被for的条件是什么?

答: 有**\_\_iter\_\_()**的容器

```
"""
```

```
#练习
```

```
lists = ["林黛玉","贾元春","贾迎春","贾探春","贾惜春"]
```

```
iterable = lists.__iter__() #获取迭代器对象
```

```
#循环获取下一个元素
```

```
while True:
```

```
 try:
```

```
 temp = iterable.__next__()
```

```
 print(temp)
```

```
 except StopIteration:
```

```
 break #遇到异常停止迭代
```

```
#练习2
```

```
dicts = {"林黛玉":101,"贾元春":102,"贾迎春":103,"贾探春":104,"贾惜春":105}
```

```
iterable = dicts.__item__()
```

```
while True:
```

```
 try:
```

```
 temp = iterable.__next__()
```

```
 print(temp,dicts[temp])
```

```
 except StopIteration:
```

```
 break
```

## 2.迭代器对象 iterator

(1)可以被next()函数调用并返回下一个值得对象

(2)语法

```
class 迭代器类名:
```

```
 def __init__(self,聚合对象):
```

```
 self.聚合对象 = 聚合对象
```

```
 def __next__(self):
```

```
 if 没有元素:
```

```
 raise StopIteration
```

```
 return 聚合对象元素
```

```
#练习 学生管理器 记录了多个学生 迭代出学生
```

```
class StudentModel:
```

```
 def __init__(self,id,name,sex):
```

```
 pass
```

```
class StudentManager:
```

```
 def __init__(self):
```

```
 self.__list_stu = []
```

```
 def insert(self,stu):
```

```
 if isinstance(stu,StudentModel):
```

```
 self.__list_stu.append(stu)
```

```
 else:
```

```
 raise ValueError()
```

```
 def __item__(self):
```

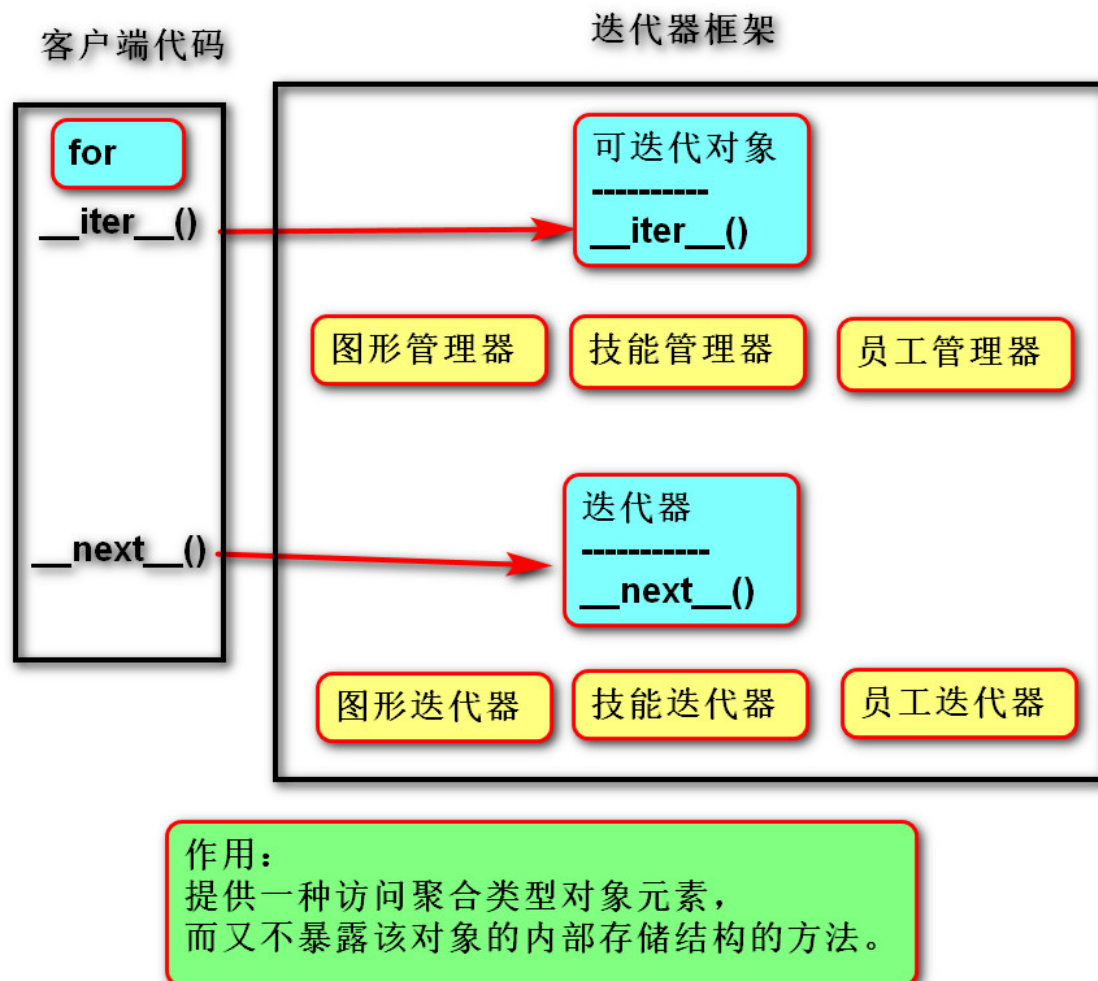
```
 return StudentIterator(self.__list_stu) #1.返回迭代器
```

```
class StudentIterator: #迭代器
 def __init__(self,lists):
 self.__lists = lists
 count = 0
 def __next__(self):
 if count<len(self.lists):
 temp =self.__lists[count]
 count += 1
 return temp
 else:
 raise StopIteration

stu = StudentModel()
stu1 = StudentModel()
stu2 = StudentModel()
manager = StudentManager()
manager.insert(stu)
manager.insert(stu1)
manager.insert(stu2)

print("-----")
#for item in manager:
iterable = manager.__iter__()#1.获取迭代器

while True:
 try:
 temp = iterable.__next__()
 print(temp)
 except StopIteration:
 break
```



## (二) 生成器

1.构成 可迭代对象+迭代器

2.定义

next (循环) 一次，计算一次，返回一次  
动态提供数据的可迭代对象，节省内存空间

3.语法

- 创建

```
def 函数名():
 yield 数据
```

- 调用

```
for i in 变量名():
 语句
```

调用生成器函数将返回一个生成器对象

4.原理

```
"""
```

生成器语法:

```
def 函数():
```

```
 yield 数据
```

```
for item in 函数名():
```

```
 语句
```

```
"""
```

#1. 定义MyRange类, 实现range的功能 过渡版

```
class MyRange:
```

```
 def __init__(self, end):
```

```
 self.end = end
```

```
 def __item__(self):
```

```
 return RangeIterator(self.end)
```

```
class RangeIterator: #迭代器
```

```
 def __init__(self, end):
```

```
 self.__begin = 0
```

```
 self.__end = end
```

```
 def __next__(self):
```

```
 if self.__begin < self.__end:
```

```
 temp = self.__begin
```

```
 self.__begin += 1
```

```
 return temp
```

```
 else:
```

```
 raise StopIteration
```

#next一次, 计算一次

```
for i in MyRange(10):
```

```
 print(i)
```

#2. 定义Myrange类, 实现range的功能 升级过渡版

```
class MyRange:
```

```
 def __init__(self, end):
```

```
 self.end = end
```

```
 def __item__(self):
```

```
 number = 0
```

```
 while number < self.end
```

```
 yield number
```

#自动生成迭代器

将下列代码改为迭代器模式

```
 number += 1
```

#3. 终极版

```
def num(value): #定义
```

```
 count = 0
```

```
 while count < value:
```

```
 yield count
```

```
 count += 1
```

```
for item in num(10): #调用
```

```
 print(item, end="")
```

#练习 学生管理器 记录了多个学生 迭代出学生 用yield

```
class StudentModel:
 def __init__(self, id, name, sex):
 pass

class StudentManager:
 def __init__(self):
 self.__list_stu = []
 def insert(self, stu):
 if isinstance(stu, StudentModel):
 self.__list_stu.append(stu)
 else:
 raise ValueError()
 def __item__(self):
 #return StudentIterator(self.__list_stu) #1. 返回迭代器
 """
 执行过程:
 1. 调用当前方法, 不执行 (内部创建迭代器对象)
 2. 调用__next__方法, 方执行
 3. 执行到yield语句, 暂时离开
 4. 再次调用__next__方法, 继续执行
 5. 重复3, 4 直到最后
 """
 count = 0
 while count < len(self.__list_stu):
 yield self.__list_stu[count] #用yield生成迭代器
 count += 1

#class StudentIterator: #迭代器
def __init__(self, lists):
self.__lists = lists
count = 0
def __next__(self):
if count < len(self.__lists):
temp = self.__lists[count]
count += 1
return temp
else:
raise StopIteration

stu = StudentModel()
stu1 = StudentModel()
stu2 = StudentModel()
manager = StudentManager()
manager.insert(stu)
manager.insert(stu1)
manager.insert(stu2)

print("-----")
#for item in manager:
```



```

iterable = manager.__iter__()#1.获取迭代器

while True:
 try:
 temp = iterable.__next__()
 print(temp)
 except StopIteration:
 break

#练习 从列表找出所有偶数
lists = [45,66,88,12,5,27]
def get_even_number(lists):
 for item in lists:
 if item %2 == 0:
 yield item

for item in get_even_number(lists):
 print(i)

#练习 定义一个生成器函数my_zip,
"""
list1 = ["林黛玉","薛宝钗"]
list2 = [102,103]
for item in zip(list1,list2): #zip 将多个列表的每个元素合并成一个元祖
 print(item)
"""

list1 = ["林黛玉","薛宝钗"]
list2 = [102,103]
def my_zip(*args):
 for i in range(len(args[0])):
 yield (item[i] for item in args)

for item in my_zip(list1,list2):
 print(item)

"""

yield作用:将下列代码改为迭代器模式的代码。

生成迭代器代码的大致规则:

1. 将yield以前的语句定义在next方法中

2. 将yield后面的数据作为next方法返回值

"""

```

#### 4.优缺点

优：节省内存

缺：不能使用索引、切片访问结果

## 5.惰性操作->立即操作

```
list_result=list(<生成器>)
```

## 6.表达式

语法

```
re = (item for item in list01 if type(item) == int) #生成器对象
```

```
for item in re:
```

```
 print(item)
```

## 7.生成器与迭代器关系

```
class 可迭代关系:
```

```
 def __iter__():
```

```
 return 迭代器对象
```

```
class 迭代器:
```

```
 def __next__():
```

调用

```
for in
```

```
面试题:
```

```
请简述，生成器与迭代器
```

```
生成器 本质就是 迭代器 + 可迭代对象。
```

```
而可迭代对象就是为了可以迭代(for)，而迭代的本质就是不断调用迭代器next方法。
```

```
生成器最重要的特点调用一次next，计算一次结果，返回一个数据，
```

```
这个过程称之为惰性操作 / 延迟操作。
```

```
在海量数据下，可以大量节省内存。
```

```
惰性操作 --> 立即操作(灵活获取结果)
```

```
list(生成器)
```

## (三) 内置函数

- 枚举函数 enumerate

遍历可迭代对象时，可以将索引与元素组合为元组

语法：

```
for 变量 in enumerate(可迭代对象):
 语句

for 索引, 元素 in enumerate(可迭代对象):
 语句
```

- zip

将多个可迭代对象中对应的元素组合成一个个元组

语法：

```
for 变量 in zip(可迭代对象1, 可迭代对象2):
 语句
```

## 十、网络编程

### (一) socket套接字编程

#### 1.套接字介绍

1. 套接字：实现网络编程进行数据传输的一种技术手段

2. Python实现套接字编程：import socket

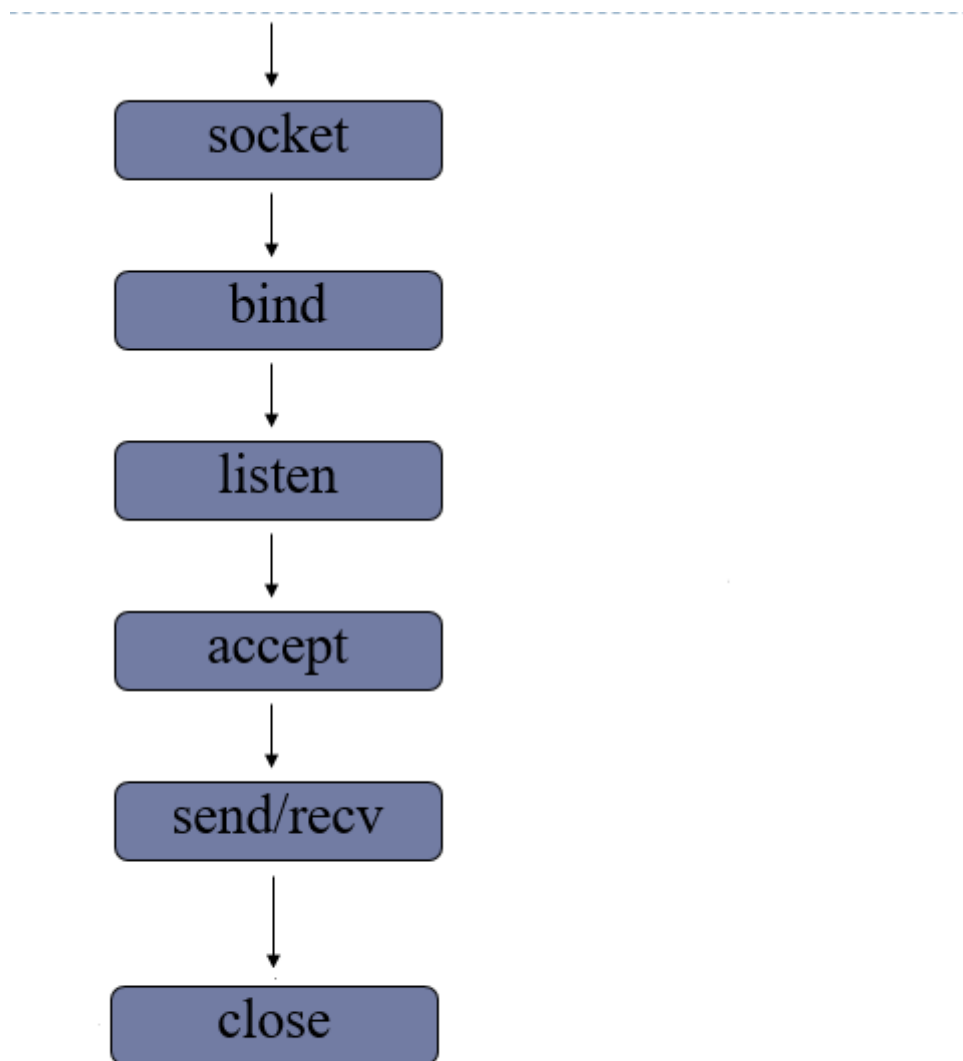
3. 套接字分类

流式套接字(SOCK\_STREAM): 以字节流方式传输数据，实现tcp网络传输方案。(面向连接--tcp协议--可靠的--流式套接字)

数据报套接字(SOCK\_DGRAM):以数据报形式传输数据，实现udp网络传输方案。(无连接--udp协议--不可靠--数据报套接字)

#### 2.tcp套接字编程

##### (1)服务端流程



如打电话流程：买电话->办卡->充话费->开机->说话->关闭

- 创建套接字

```
sockfd=socket.socket(socket_family=AF_INET,socket_type=SOCK_STREAM,proto=0)
```

功能：创建套接字

参数：

**socket\_family** 网络地址类型 AF\_INET表示ipv4 AF\_INET6表示ipv6

**socket\_type** 套接字类型 SOCK\_STREAM(流式) SOCK\_DGRAM(数据报)

**proto** 通常为0 选择子协议

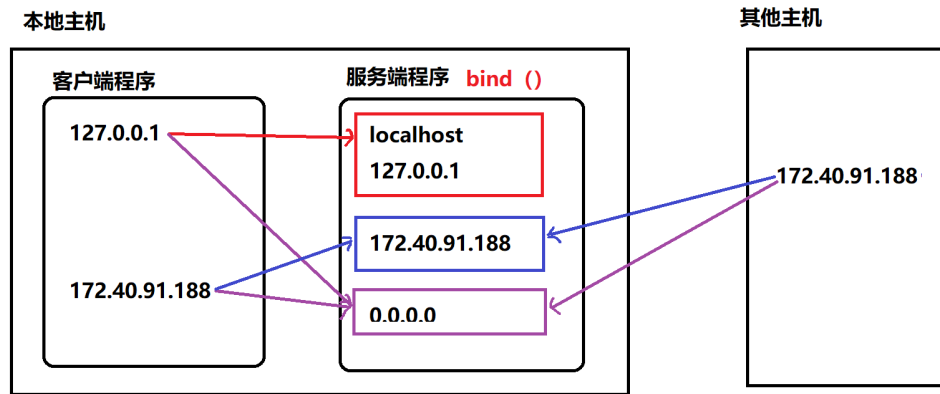
返回值：套接字对象

- 绑定地址

本地地址：'localhost','127.0.0.1'

网络地址：'172.40.91.185' ifconfig得到的，即自己网卡的ip

自动获取地址：'0.0.0.0'



`sockfd.bind(addr)`

功能： 绑定本机网络地址

参数： 二元组 (ip,port) ('0.0.0.0',8888)

- 设置监听

`sockfd.listen(n)`

功能： 将套接字设置为监听套接字，确定监听队列大小

参数： 监听队列大小

- 等待处理客户端连接请求

`connfd,addr = sockfd.accept()`

功能： 阻塞等待处理客户端请求

返回值： `connfd` 客户端连接套接字 代表客户端对象  
`addr` 连接的客户端地址

- 消息收发

`data = connfd.recv(buffer size)`

功能： 接受客户端消息

参数： 每次最多接收消息的大小

返回值： 接收到的内容

`n = connfd.send(data)`

功能： 发送消息

参数： 要发送的内容 `bytes`格式 字节串

返回值： 发送的字节数

- 关闭套接字

`sockfd.close()`

功能： 关闭套接字

```
#服务端
import socket
#创建tcp套接字
```

```

sockfd = socket.socket(socket.AF_INET,
 socket.SOCK_STREAM)

#绑定地址
sockfd.bind(("127.0.0.1", 8080))

#设置监听
sockfd.listen(6)

#阻塞等待处理连接
print("等待连接")
connfd, addr = socket.accept()
print("已连接", addr) #打印连接的ip

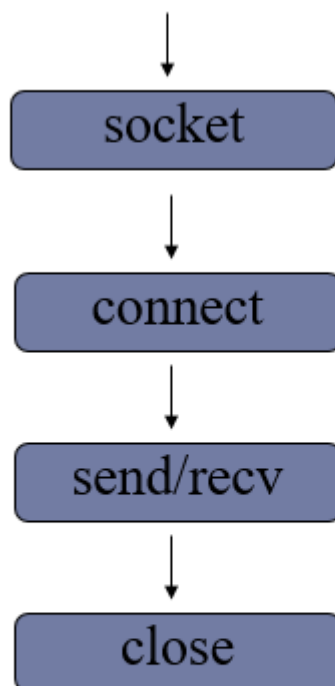
#收发消息
data = connfd.recv(1024)

n = connfd.send(b"Thanks") #发送字节串

#关闭套接字
connfd.close()
sockfd

```

## (2)客户端流程



### 1)创建套接字

注意:只有相同类型的套接字才能进行通信

### 2)请求连接

```
sockfd.connect(server_addr)
```

功能：连接服务器

参数：元组 服务器地址

### 3)收发消息

注意：防止两端都阻塞，recv send要配合

### 4)关闭套接字

```
#客户端
from socket import #
sockfd = socket() #tcp

sockfd.connect((192.168.65.1,8888))#连接

data = input("") #发消息
sockfd.send(data.encode()) #发字节码
data = sockfd.recv(1024)
print(data.decode())

sockfd.close()
```

### (3)tcp 套接字数据传输特点

- tcp连接中当一端退出，另一端如果阻塞在recv，此时recv会立即返回一个空字符串。
- tcp连接中如果一端已经不存在，仍然试图通过send发送则会产生BrokenPipeError
- 一个监听套接字可以同时连接多个客户端，也能够重复被连接

```
#服务端
from socket import *
sockfd = socket()

sockfd.connect(("127.0.0.1",8888))

while True:
 data =input("请输入: ")
 if not data:
 break
 sockfd.send(data.encode())
 print(sockfd.recv(1024).decode())

sockfd.close()

import socket

创建服务器
sockfd = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

sockfd.bind(("127.0.0.1", 8888))

sockfd.listen(5) # 监听
while True:
 print("等待请求")
```

```

try:
 connfd,addr=sockfd.accept() #等待请求
except KeyboardInterrupt:
 print("服务器退出")
except Exception as e:
 print(e)
 continue
while True:
 data = connfd.recv(1024) #接受消息
 if not data:
 break
 print(data.decode())
 temp = '{}用户, 你好'
 connfd.send(temp.encode())
 connfd.close()

sockfd.close()

```

```

#传输文件
#server
from socket import *

sockfd =socket()
sockfd.bind(("127.0.0.1",8888))
sockfd.listen(5)
connfd,addr = s.accept()
with open("1d.jpg","wb") as fd:
 while True:
 data = c.recv(1024)
 if not data:
 break
 fd.write(data)
connfd.close()
sockfd.close()

#client
from socket import *
sockfd = socket()
sockfd.connect(("127.0.0.1",8888))

fd = open("jpg.jpg","rb")
while True:
 data = fd.read(1024)
 if not data:
 break
 s.send(data)

fd.close()
sockfd.close()

```



#### (4)网络收发缓冲区

1. 网络缓冲区有效的协调了消息的收发速度
2. send和recv实际是向缓冲区发送接收消息，当缓冲区不为空recv就不会阻塞。

#### (5)tcp粘包

原因：tcp以字节流方式传输，没有消息边界。多次发送的消息被一次接收，此时就会形成粘包。

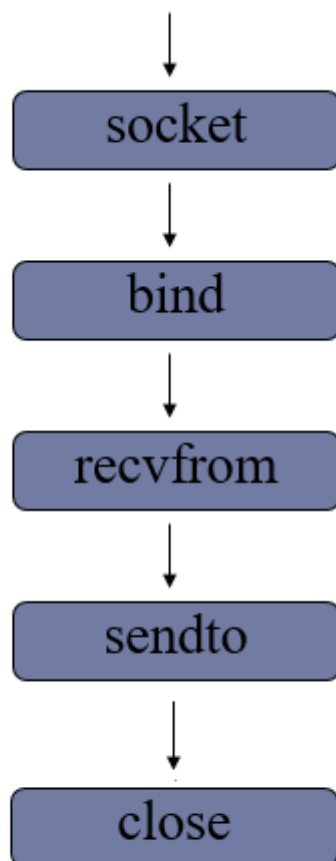
影响：如果每次发送内容是一个独立的含义，需要接收端独立解析此时粘包会有影响。

处理方法

1. 人为的添加消息边界
2. 控制发送速度

### 3.UDP套接字编程

#### 1)服务端流程



- 创建数据报套接字

```
sockfd = socket(AF_INET, SOCK_DGRAM)
```

- 绑定地址

```
sockfd.bind(addr)
```

- 消息收发

```
data,addr = sockfd.recvfrom(bufferize)
```

功能： 接收UDP消息

参数： 每次最多接收多少字节

返回值： **data** 接收到的内容

**addr** 消息发送方地址

```
n = sockfd.sendto(data,addr)
```

功能： 发送UDP消息

参数： **data** 发送的内容 **bytes**格式

**addr** 目标地址

返回值： 发送的字节数

- 关闭套接字

```
sockfd.close()
```

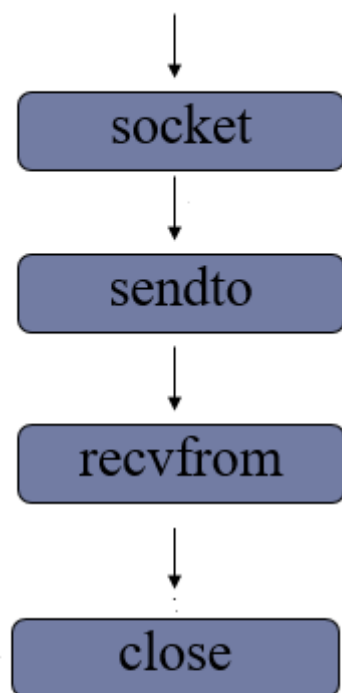
```
#server
from socket import *
sockfd = socket(AF_INET,SOCK_DGRAM)

sockfd.bind(("127.0.0.1",8888))

while True:
 data,addr = sockfd.recvfrom(1024)
 print(data)
 sockfd.sendto(b"Thanks",addr)

sockfd.close()
```

## 2)客户端流程



1. 创建套接字

2. 收发消息

3. 关闭套接字

```
from socket import *
ADDR = ("127.0.0.1", 8888)
sockfd = socket(AF_INET, SOCK_DGRAM)
while True:
 data = input("123")
 if not data:
 break
 sockfd.sendto(data.encode(), ADDR)

 n, addr = sockfd.recvfrom(1024)
 print(n)

sockfd.close()
```

## 总结：tcp套接字和udp套接字编程区别

1. 流式套接字是以字节流方式传输数据，数据报套接字以数据报形式传输
2. tcp套接字会有粘包，udp套接字有消息边界不会粘包
3. tcp套接字保证消息的完整性，udp套接字则不能
4. tcp套接字依赖listen accept建立连接才能收发消息，udp套接字则不需要
5. tcp套接字使用send, recv收发消息，udp套接字使用sendto, recvfrom

#查字典

#服务端 逻辑和数据处理

from socket import \*

```
def get_mean(word):
 with open("dict.txt","r") as fd:
 for item in fd:
 temp = item.strip("\n").split(" ")
 if word == temp[0]:
 return (temp[0]+" "+temp[-1])
 elif word < temp[0]:
 return "没找到"
```

sockfd = socket(AF\_INET,SOCK\_DGRAM)

sockfd.bind(("127.0.0.1",8888))

while True:

```
 data,addr = sockfd.recvfrom(1024)
 result=get_mean(data.decode())
 sockfd.sendto(result.encode(),addr)
```

sockfd.close()

#客户端 发送请求

from socket import \*

sockfd = socket(AF\_INET,SOCK\_DGRAM)

ADDR=("127.0.0.1",8888)

while True:

```
 data = input("请输入单词")
 if not data:
 break
 sockfd.sendto(data.encode(),ADDR)
```

```
 n,addr = sockfd.recvfrom(1024)
 print(n.decode())
```

sockfd.close()

#广播

```

"""
1. 创建udp套接字
2. 设置套接字可以发送接收广播(setsockopt)
3. 选择接收的端口
4. 接收广播
"""

#server
from socket import *
ADDR=("172.40.91.255",9999) #"172.40.91.255" ifconfig查询
sockfd = socket(AF_INET,SOCK_DGRAM)

sockfd.setsockopt(SOL_SOCKET,SO_REUSEADDR,1)
data=" "
while True:
 sockfd.sendto(data.encode(),ADDR)
#client
from socket import *
sockfd = socket(AF_INET,SOCK_DGRAM)

#设置套接字接受广播
sockfd.setsockopt(SOL_SOCKET,SO_BROADCAST,1)

sockfd.bind("0.0.0.0",9999)

while True:
 msg,addr = sockfd.recvfrom(1024)
 print(msg.decode())

```

## 4.socket套接字属性

- 【1】 sockfd.type 套接字类型
- 【2】 sockfd.family 套接字地址类型
- 【3】 sockfd.getsockname() 获取套接字绑定地址
- 【4】 sockfd.fileno() 获取套接字的文件描述符
- 【5】 sockfd.getpeername() 获取连接套接字客户端地址
- 【6】 sockfd.setsockopt(level,option,value)
  - 功能：设置套接字选项
  - 参数： level 选项类别 SOL\_SOCKET
  - option 具体选项内容
  - value 选项值

SOL\_SOCKET 的option选项

| 选项                           | 意义                                                                                                    | 期望值                       |
|------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------|
| <code>SO_BINDTODEVICE</code> | 可以使 <code>socket</code> 只在某个特殊的网络接口（网卡）有效。也许不能是移动便携设备                                                 | 一个字符串给出设备的名称或者一个空字符串返回默认值 |
| <code>SO_BROADCAST</code>    | 允许广播地址发送和接收信息包。只对 <code>UDP</code> 有效。如何发送和接收广播信息包                                                    | 布尔型整数                     |
| <code>SO_DONTROUTE</code>    | 禁止通过路由器和网关往外发送信息包。这主要是为了安全而用在以太网上 <code>UDP</code> 通信的一种方法。不管目的地使用什么 <code>IP</code> 地址，都可以防止数据离开本地网络 | 布尔型整数                     |
| <code>SO_KEEPALIVE</code>    | 可以使 <code>TCP</code> 通信的信息包保持连续性。这些信息包可以在没有信息传输的时候，使通信的双方确定连接是保持的                                     | 布尔型整数                     |
| <code>SO_OOBINLINE</code>    | 可以把收到的不正常数据看成是正常的，也就是说会通过一个标准的对 <code>recv()</code> 的调用来接收这些数据                                        | 布尔型整数                     |
| <code>SO_REUSEADDR</code>    | 当 <code>socket</code> 关闭后，本地端用于该 <code>socket</code> 的端口号立刻就可以被重用。通常来说，只有经过系统定义一段时间后，才能被重用。           | 布尔型整数                     |

## (二)struct模块进行数据打包

1. 原理： 将一组简单数据进行打包，转换为bytes格式发送。或者将一组bytes格式数据，进行解析。
2. 接口使用

`Struct(fmt)`

功能： 生成结构化对象

参数： `fmt` 定制的数据结构

`st.pack(v1,v2,v3,...)`

功能： 将一组数据按照指定格式打包转换为bytes

参数：要打包的数据  
返回值： `bytes` 字节串

`st.unpack(bytes_data)`

功能： 将`bytes`字节串按照指定的格式解析

参数： 要解析的字节串

返回值： 解析后的内容

`struct.pack(fmt,v1,v2,v3...)`

`struct.unpack(fmt,bytes_data)`

说明： 可以使用`struct`模块直接调用`pack` `unpack`。此时这两函数第一个参数传入`fmt`。其他用法功能相同

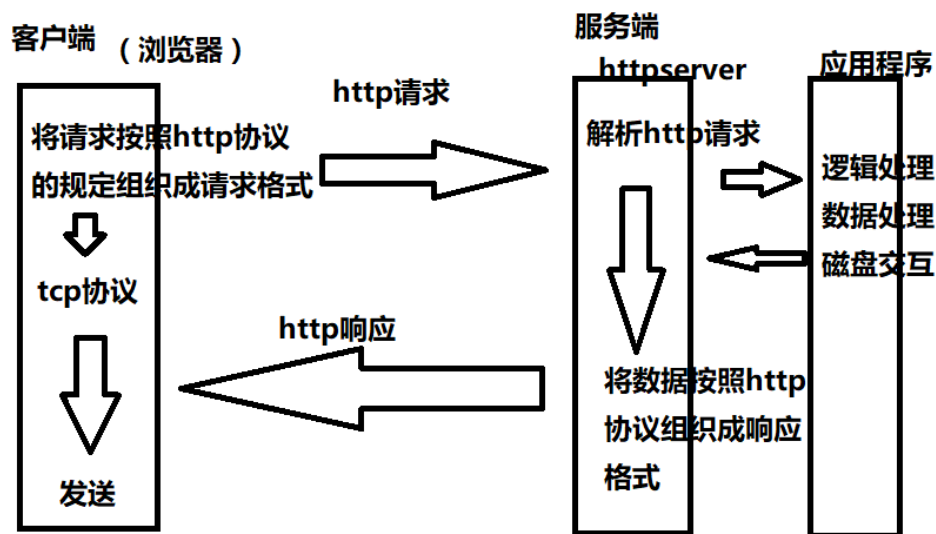
## 1.HTTP传输

### 1)HTTP协议（超文本传输协议）

1. 用途： 网页获取，数据的传输

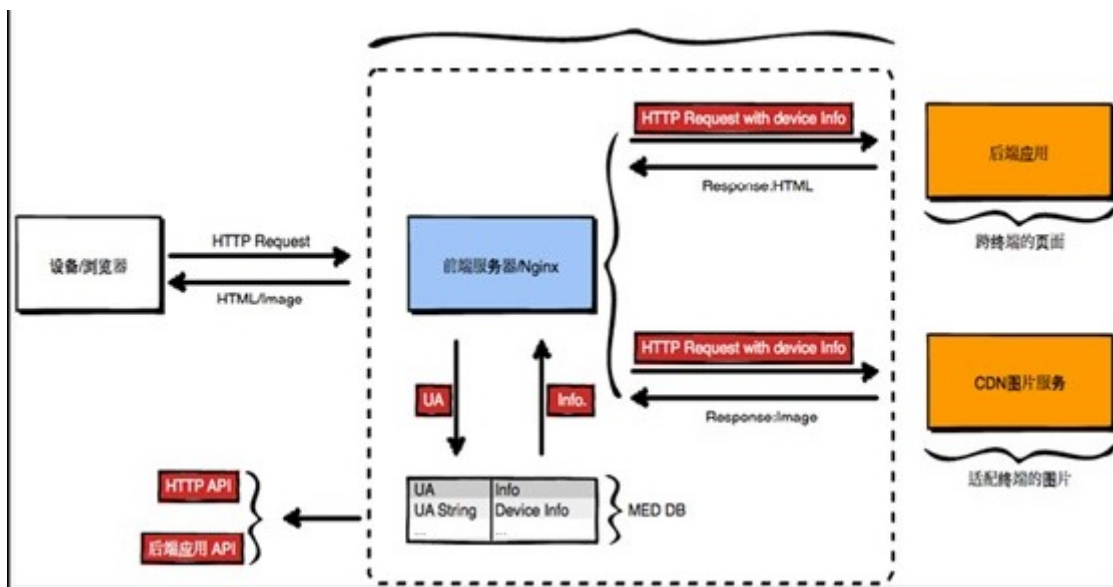
2. 特点

- 应用层协议，传输层使用`tcp`传输
- 简单，灵活，很多语言都有HTTP专门接口
- 无状态，协议不记录传输内容
- `http1.1` 支持持久连接，丰富了请求类型



### 3. 网页请求过程

1. 客户端（浏览器）通过`tcp`传输，发送`http`请求给服务端
2. 服务端接收到`http`请求后进行解析
3. 服务端处理请求内容，组织响应内容
4. 服务端将响应内容以`http`响应格式发送给浏览器
5. 浏览器接收到响应内容，解析展示



## 2.HTTP请求 (request)

- 请求行：具体的请求类别和请求内容

|      |      |          |
|------|------|----------|
| GET  | /    | HTTP/1.1 |
| 请求类别 | 请求内容 | 协议版本     |

请求类别：每个请求类别表示要做不同的事情

GET：获取网络资源  
 POST：提交一定的信息，得到反馈  
 HEAD：只获取网络资源的响应头  
 PUT：更新服务器资源  
 DELETE：删除服务器资源  
 CONNECT  
 TRACE：测试  
 OPTIONS：获取服务器性能信息

- 请求头：对请求的进一步解释和描述

Accept-Encoding: gzip

- 空行
- 请求体: 请求参数或者提交内容

## 3.http响应 (response)

1. 响应格式：响应行，响应头，空行，响应体

- 响应行：反馈基本的响应情况

|          |     |      |
|----------|-----|------|
| HTTP/1.1 | 200 | OK   |
| 版本信息     | 响应码 | 附加信息 |

响应码：



1xx 提示信息，表示请求被接收  
2xx 响应成功  
3xx 响应需要进一步操作，重定向  
4xx 客户端错误  
5xx 服务器错误

- 响应头：对响应内容的描述

Content-Type: text/html

- 响应体：响应的主体内容信息

## 十一、并发编程

### (一) 多任务编程

1. 意义：充分利用计算机CPU的多核资源，同时处理多个应用程序任务，以此提高程序的运行效率。
2. 实现方案：多进程，多线程
3. 并行与并发

并发：同时处理多个任务，内核在任务间不断地切换达到好像多个任务被同时执行的效果，实际每个时刻只有一个任务占有内核。

并行：多个任务利用计算机多核资源在同时执行，此时多个任务为并行关系

### (二) 进程 (process)

#### 进程理论基础

1. 定义：程序在计算机中的一次运行。

- 程序是一个可执行的文件，是静态的占有磁盘。
- 进程是一个动态的过程描述，占有计算机运行资源，有一定的生命周期。

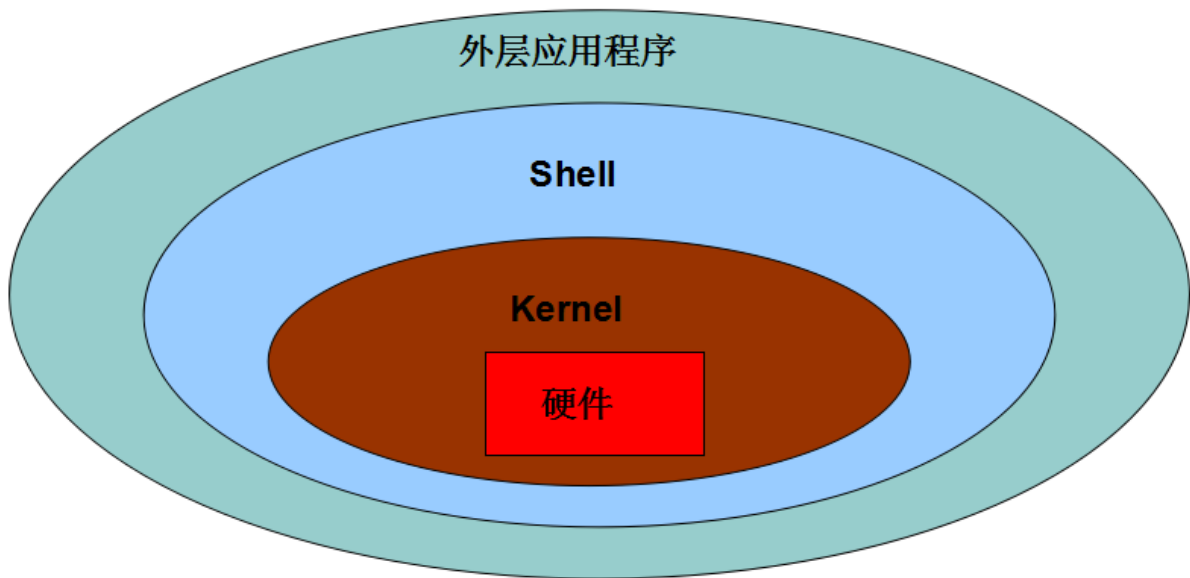
2. 系统中如何产生一个进程

【1】用户空间通过调用程序接口或者命令发起请求

【2】操作系统接收用户请求，开始创建进程

【3】操作系统调配计算机资源，确定进程状态等

【4】操作系统将创建的进程提供给用户使用



### 3. 进程基本概念

- cpu时间片：如果一个进程占有cpu内核则称这个进程在cpu时间片上。
- PCB(进程控制块)：在内存中开辟的一块空间，用于存放进程的基本信息，也用于系统查找识别进程。
- 进程ID (PID)：系统为每个进程分配的一个大于0的整数，作为进程ID。每个进程ID不重复。

Linux查看进程ID：ps -aux

- 父子进程：系统中每一个进程(除了系统初始化进程)都有唯一的父进程，可以有0个或多个子进程。父子进程关系便于进程管理。

查看进程树：pstree

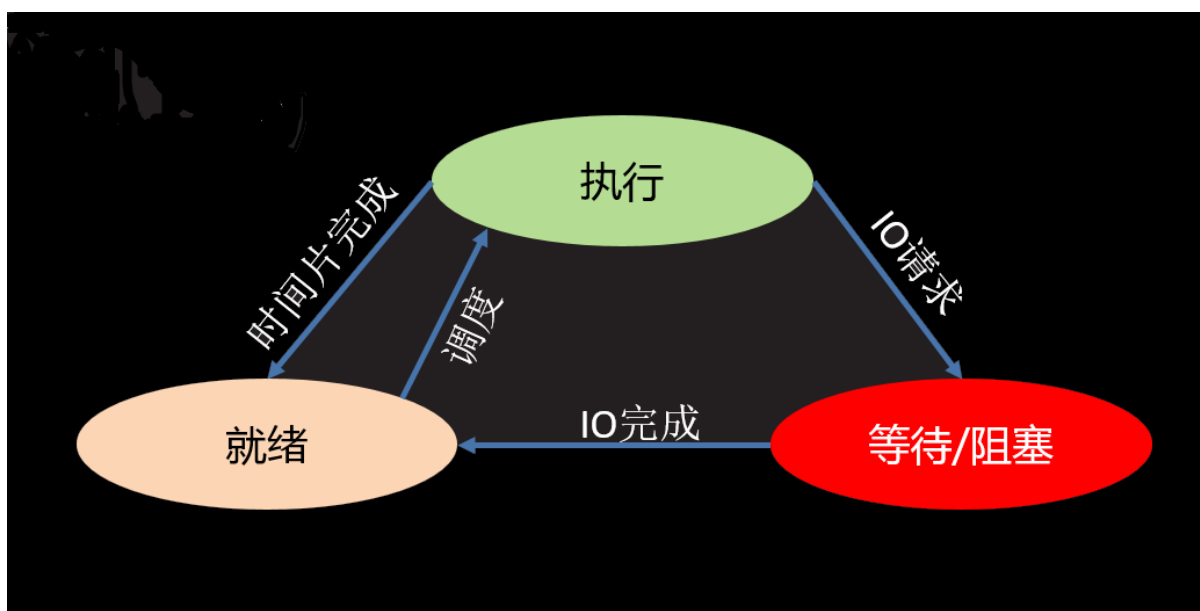
#### • 进程状态

##### ◦ 三态

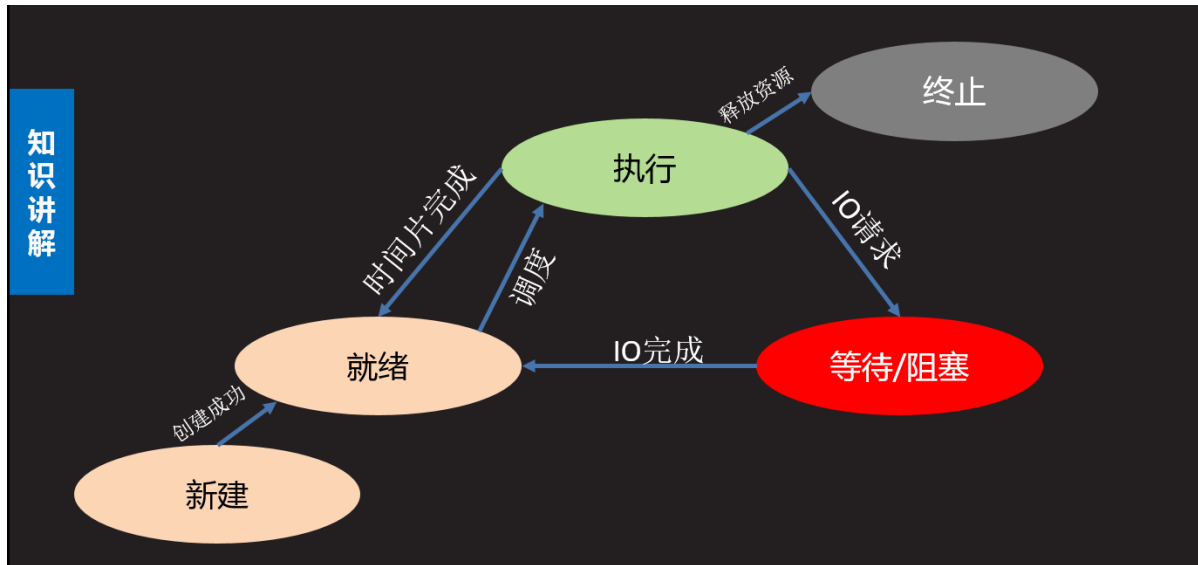
就绪态：进程具备执行条件，等待分配cpu资源

运行态：进程占有cpu时间片正在运行

等待态：进程暂时停止运行，让出cpu



- 五态 (在三态基础上增加新建和终止)  
建： 创建一个进程，获取资源的过程  
终止： 进程结束，释放资源的过程



- 状态查看命令： `ps -aux --> STAT列`

S 等待态  
R 执行态  
D 等待态  
T 等待态  
Z 僵尸

< 有较高优先级  
N 优先级较低  
+ 前台进程  
s 回话组组长  
l 有多线程的

- 进程的运行特征
  - 【1】多进程可以更充分使用计算机多核资源
  - 【2】进程之间的运行互不影响，各自独立
  - 【3】每个进程拥有独立的空间，各自使用自己空间资源

#### 面试要求

1. 什么是进程，进程和程序有什么区别
2. 进程有哪些状态，状态之间如何转化

## (三) 基于fork的多进程编程

### 1.fork使用

```
pid = os.fork()
```

功能：创建新的进程

返回值：整数，如果创建进程失败返回一个负数，如果成功则在原有进程中返回新进程的PID，在新进程中返回0

#### 注意

- 子进程会复制父进程全部内存空间，从fork下一句开始执行。

- 父子进程各自独立运行，运行顺序不一定。
- 利用父子进程fork返回值的区别，配合if结构让父子进程执行不同的内容几乎是固定搭配。
- 父子进程有各自特有特征比如PID PCB 命令集等。
- 父进程fork之前开辟的空间子进程同样拥有，父子进程对各自空间的操作不会相互影响。

```

"""
fork.py fork进程创建演示
"""

import os
from time import sleep

创建子进程
pid = os.fork()

if pid < 0:
 print("Create process failed")
elif pid == 0:
 # 只有子进程执行
 sleep(3)
 print("The new process")
else:
 # 只有父进程执行
 sleep(4)
 print("The old process")

父子进程都执行
print("process test over")

```

## 2.进程相关函数

**代码示例: day7/get\_pid.py**

**代码示例: day7/exit.py**

os.getpid()  
 功能： 获取一个进程的PID值  
 返回值： 返回当前进程的PID

os.getppid()  
 功能： 获取父进程的PID号  
 返回值： 返回父进程PID

os.\_exit(status)  
 功能: 结束一个进程  
 参数: 进程的终止状态

sys.exit([status])  
 功能： 退出进程  
 参数： 整数 表示退出状态  
 字符串 表示退出时打印内容

## 孤儿和僵尸

1. 孤儿进程：父进程先于子进程退出，此时子进程成为孤儿进程。

特点：孤儿进程会被系统进程收养，此时系统进程就会成为孤儿进程新的父进程，孤儿进程退出该进程会自动处理。

2. 僵尸进程：子进程先于父进程退出，父进程又没有处理子进程的退出状态，此时子进程就会称为僵尸进程。

特点：僵尸进程虽然结束，但是会存留部分PCB在内存中，大量的僵尸进程会浪费系统的内存资源。

3. 如何避免僵尸进程产生

- 使用wait函数处理子进程退出

``

```
pid,status = os.wait()
```

功能：在父进程中阻塞等待处理子进程退出

返回值：pid 退出的子进程的PID

status 子进程退出状态

...

- 创建二级子进程处理僵尸

【1】父进程创建子进程，等待回收子进程

【2】子进程创建二级子进程然后退出

【3】二级子进程称为孤儿，和原来父进程一同执行事件

```
pid = os.fork()
if pid == 0:
 p = os.fork() # 创建二级子进程
 if p == 0:
 f1()
 else:
 os._exit(0) # 一级子进程退出
else:
 os.wait() # 等待回收一级子进程
 f2()
```

- 通过信号处理子进程退出

原理：子进程退出时会发送信号给父进程，如果父进程忽略子进程信号，则系统就会自动处理子进程退出。

方法：使用signal模块在父进程创建子进程前写如下语句：

```
import signal
signal.signal(signal.SIGCHLD,signal.SIG_IGN)
```

特点：非阻塞，不会影响父进程运行。可以处理所有子进程退出

## 群聊聊天室

功能：类似qq群功能

- 【1】有人进入聊天室需要输入姓名，姓名不能重复
- 【2】有人进入聊天室时，其他人会收到通知：xxx 进入了聊天室
- 【3】一个人发消息，其他人会收到：xxx：xxxxxxxxxxxx
- 【4】有人退出聊天室，则其他人也会收到通知:xxx退出了聊天室
- 【5】扩展功能：服务器可以向所有用户发送公告:管理员消息：xxxxxxxxxx

## (四) multiprocessing 模块创建进程

### 1.进程创建方法

#### 1. 流程特点

- 【1】将需要子进程执行的事件封装为函数
- 【2】通过模块的Process类创建进程对象，关联函数
- 【3】可以通过进程对象设置进程信息及属性
- 【4】通过进程对象调用start启动进程
- 【5】通过进程对象调用join回收进程

#### 2. 基本接口使用

##### Process()

功能：创建进程对象

参数：**target** 绑定要执行的目标函数

**args** 元组，用于给**target**函数位置传参

**kwargs** 字典，给**target**函数键值传参

##### p.start()

功能：启动进程

注意:启动进程此时target绑定函数开始执行，该函数作为子进程执行内容，此时进程真正被创建

##### p.join([timeout])

功能：阻塞等待回收进程

参数：超时时间

#### 注意

- 使用multiprocessing创建进程同样是子进程复制父进程空间代码段，父子进程运行互不影响。
- 子进程只运行target绑定的函数部分，其余内容均是父进程执行内容。
- multiprocessing中父进程往往只用来创建子进程回收子进程，具体事件由子进程完成。
- multiprocessing创建的子进程中无法使用标准输入

#### 3. 进程对象属性

p.name 进程名称

p.pid 对应子进程的PID号

p.is\_alive() 查看子进程是否在生命周期

p.daemon 设置父子进程的退出关系

- 如果设置为True则子进程会随父进程的退出而结束
- 要求必须在start()前设置
- 如果daemon设置成True 通常就不会使用 join()

## 2.自定义进程类

### 1. 创建步骤

- 【1】 继承Process类
- 【2】 重写 `__init__` 方法添加自己的属性，使用super()加载父类属性
- 【3】 重写run()方法

### 2. 使用方法

- 【1】 实例化对象
- 【2】 调用start自动执行run方法
- 【3】 调用join回收线程

## 3.进程池实现

### 1. 必要性

- 【1】 进程的创建和销毁过程消耗的资源较多
- 【2】 当任务量众多，每个任务在很短时间内完成时，需要频繁的创建和销毁进程。此时对计算机压力较大
- 【3】 进程池技术很好的解决了以上问题。

### 2. 原理

创建一定数量的进程来处理事件，事件处理完进程不退出而是继续处理其他事件，直到所有事件全都处理完毕统一销毁。增加进程的重复利用，降低资源消耗。

### 3. 进程池实现

#### 【1】 创建进程池对象，放入适当的进程

```
from multiprocessing import Pool
```

```
Pool(processes)
```

功能： 创建进程池对象

参数： 指定进程数量，默认根据系统自动判定

#### 【2】 将事件加入进程池队列执行

```
pool.apply_async(func, args, kwds)
```

功能： 使用进程池执行 func事件

参数： func 事件函数

args 元组 给func按位置传参

kwds 字典 给func按照键值传参

返回值： 返回函数事件对象

#### 【3】 关闭进程池

```
pool.close()
```

功能： 关闭进程池

## 【4】回收进程池中进程

```
pool.join()
```

功能：回收进程池中进程

## (五) 进程间通信 (IPC)

1. 必要性：进程间空间独立，资源不共享，此时在需要进程间数据传输时就需要特定的手段进行数据通信。

2. 常用进程间通信方法

管道 消息队列 共享内存 信号 信号量 套接字

### 1.管道通信(Pipe)

1. 通信原理

在内存中开辟管道空间，生成管道操作对象，多个进程使用同一个管道对象进行读写即可实现通信

2. 实现方法

```
from multiprocessing import Pipe
```

```
fd1,fd2 = Pipe(duplex = True)
```

功能：创建管道

参数：默认表示双向管道

如果为False 表示单向管道

返回值：表示管道两端的读写对象

如果是双向管道均可读写

如果是单向管道fd1只读 fd2只写

```
fd.recv()
```

功能：从管道获取内容

返回值：获取到的数据

```
fd.send(data)
```

功能：向管道写入内容

参数：要写入的数据

### 2.消息队列

1.通信原理

在内存中建立队列模型，进程通过队列将消息存入，或者从队列取出完成进程间通信。

2. 实现方法

```
from multiprocessing import Queue
```

```
q = Queue(maxsize=0)
```

功能：创建队列对象

参数：最多存放消息个数

返回值：队列对象

```
q.put(data,[block,timeout])
```



功能：向队列存入消息  
参数：data 要存入的内容  
block 设置是否阻塞 **False**为非阻塞  
timeout 超时检测

q.get([block,timeout])  
功能：从队列取出消息  
参数：block 设置是否阻塞 **False**为非阻塞  
timeout 超时检测  
返回值： 返回获取到的内容

q.full() 判断队列是否为满  
q.empty() 判断队列是否为空  
q.qsize() 获取队列中消息个数  
q.close() 关闭队列

### 3.共享内存

1. 通信原理：在内中开辟一块空间，进程可以写入内容和读取内容完成通信，但是每次写入内容会覆盖之前内容。
2. 实现方法

| Type code | C Type         | Python Type       | Minimum size in bytes |
|-----------|----------------|-------------------|-----------------------|
| 'c'       | char           | character         | 1                     |
| 'b'       | signed char    | int               | 1                     |
| 'B'       | unsigned char  | int               | 1                     |
| 'u'       | Py_UNICODE     | Unicode character | 2 (see note)          |
| 'h'       | signed short   | int               | 2                     |
| 'H'       | unsigned short | int               | 2                     |
| 'i'       | signed int     | int               | 2                     |
| 'I'       | unsigned int   | long              | 2                     |
| 'l'       | signed long    | int               | 4                     |
| 'L'       | unsigned long  | long              | 4                     |
| 'f'       | float          | float             | 4                     |
| 'd'       | double         | float             | 8                     |

```
from multiprocessing import Value,Array
```

obj = Value(ctype,data)  
功能： 开辟共享内存  
参数： ctype 表示共享内存空间类型 **'i'** **'f'** **'c'**  
data 共享内存空间初始数据  
返回值：共享内存对象

obj.value 对该属性的修改查看即对共享内存读写

obj = Array(ctype,data)  
功能： 开辟共享内存空间  
参数： ctype 表示共享内存数据类型

**data** 整数则表示开辟空间的大小，其他数据类型表示开辟空间存放的初始化数据  
返回值：共享内存对象

**Array**共享内存读写： 通过遍历**obj**可以得到每个值，直接可以通过索引序号修改任意值。

\* 可以使用**obj.value**直接打印共享内存中的字节串

## 4.信号量（信号灯集）

### 1. 通信原理

给定一个数量对多个进程可见。多个进程都可以操作该数量增减，并根据数量值决定自己的行为。

### 2. 实现方法

```
from multiprocessing import Semaphore
```

```
sem = Semaphore(num)
```

功能： 创建信号量对象

参数： 信号量的初始值

返回值： 信号量对象

**sem.acquire()** 将信号量减1 当信号量为0时阻塞

**sem.release()** 将信号量加1

**sem.get\_value()** 获取信号量数量

## （六）线程编程（Thread）

### 1.线程基本概念

#### 1. 什么是线程

- 【1】 线程被称为轻量级的进程
- 【2】 线程也可以使用计算机多核资源，是多任务编程方式
- 【3】 线程是系统分配内核的最小单元
- 【4】 线程可以理解为进程的分支任务

#### 2. 线程特征

- 【1】 一个进程中可以包含多个线程
- 【2】 线程也是一个运行行为，消耗计算机资源
- 【3】 一个进程中的所有线程共享这个进程的资源
- 【4】 多个线程之间的运行互不影响各自运行
- 【5】 线程的创建和销毁消耗资源远小于进程
- 【6】 各个线程也有自己的ID等特征

### 2.threading模块创建线程

- 【1】 创建线程对象

```
from threading import Thread
```

```
t = Thread()
```

功能：创建线程对象

参数：target 绑定线程函数

args 元组 给线程函数位置传参

kwargs 字典 给线程函数键值传参

## 【2】启动线程

```
t.start()
```

## 【3】回收线程

```
t.join([timeout])
```

## 3.线程对象属性

t.name 线程名称

t.setName() 设置线程名称

t.getName() 获取线程名称

t.is\_alive() 查看线程是否在生命周期

t.daemon 设置主线程和分支线程的退出关系

t.setDaemon() 设置daemon属性值

t.isDaemon() 查看daemon属性值

daemon为True时主线程退出分支线程也退出。要在start前设置，通常不和join一起使用。

## 4.自定义线程类

### 1. 创建步骤

【1】继承Thread类

【2】重写\_\_init\_\_方法添加自己的属性，使用super()加载父类属性

【3】重写run()方法

### 2. 使用方法

【1】实例化对象

【2】调用start自动执行run方法

【3】调用join回收线程

## (七) 同步互斥

### 1.线程间通信方法

#### 1. 通信方法

线程间使用全局变量进行通信

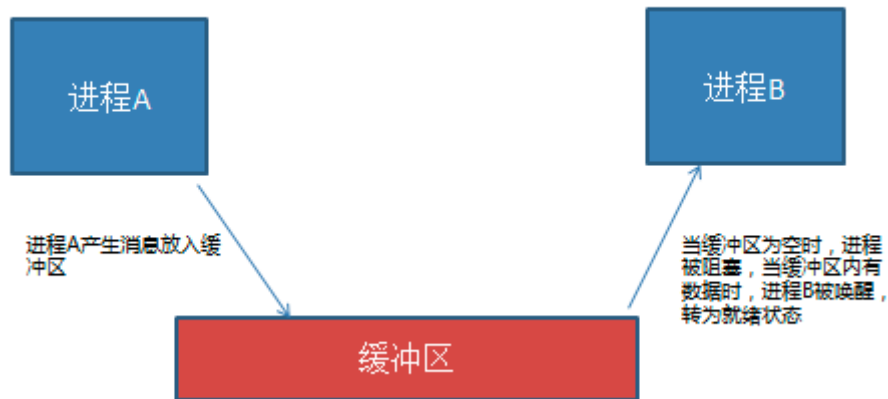
#### 2. 共享资源争夺

- 共享资源：多个进程或者线程都可以操作的资源称为共享资源。对共享资源的操作代码段称为临界区。

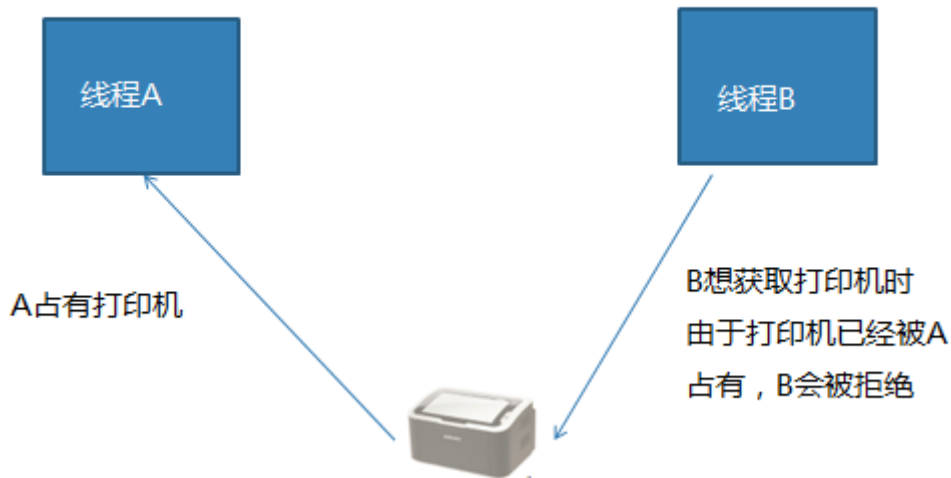
- 影响：对共享资源的无序操作可能会带来数据的混乱，或者操作错误。此时往往需要同步互斥机制协调操作顺序。

### 3. 同步互斥机制

同步：同步是一种协作关系，为完成操作，多进程或者线程间形成一种协调，按照必要的步骤有序执行操作。



互斥：互斥是一种制约关系，当一个进程或者线程占有资源时会进行加锁处理，此时其他进程线程就无法操作该资源，直到解锁后才能操作。



## 2. 线程同步互斥方法

线程Event

代码示例: `day10/thread_event.py`

```
from threading import Event

e = Event() 创建线程event对象

e.wait([timeout]) 阻塞等待e被set

e.set() 设置e, 使wait结束阻塞

e.clear() 使e回到未被设置状态

e.is_set() 查看当前e是否被设置
```

## 线程锁 Lock

代码示例: *day10/thread\_event.py*

```
from threading import Lock

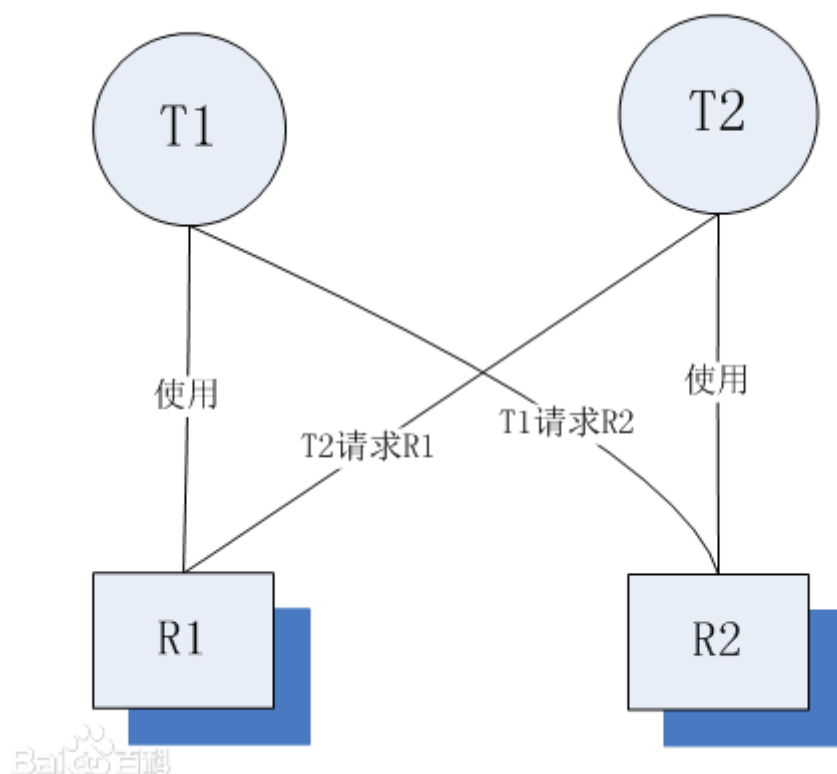
lock = Lock() 创建锁对象
lock.acquire() 上锁 如果lock已经上锁再调用会阻塞
lock.release() 解锁

with lock: 上锁
 ...
 ...
 with代码块结束自动解锁
```

## 3.死锁及其处理

### 1. 定义

死锁是指两个或两个以上的线程在执行过程中, 由于竞争资源或者由于彼此通信而造成的一种阻塞的现象, 若无外力作用, 它们都将无法推进下去。此时称系统处于死锁状态或系统产生了死锁。



## 2. 死锁产生条件

代码示例: `day10/dead_lock.py`

死锁发生的必要条件

- 互斥条件：指线程对所分配到的资源进行排它性使用，即在一段时间内某资源只由一个进程占用。如果此时还有其它进程请求资源，则请求者只能等待，直至占有资源的进程用毕释放。
- 请求和保持条件：指线程已经保持至少一个资源，但又提出了新的资源请求，而该资源已被其它进程占有，此时请求线程阻塞，但又对自己已获得的其它资源保持不放。
- 不剥夺条件：指线程已获得的资源，在未使用完之前，不能被剥夺，只能在使用完时由自己释放。通常CPU内存资源是可以被系统强行调配剥夺的。
- 环路等待条件：指在发生死锁时，必然存在一个线程——资源的环形链，即进程集合  $\{T_0, T_1, T_2, \dots, T_n\}$  中的  $T_0$  正在等待一个  $T_1$  占用的资源； $T_1$  正在等待  $T_2$  占用的资源，……， $T_n$  正在等待已被  $T_0$  占用的资源。

死锁的产生原因

简单来说造成死锁的原因可以概括成三句话：

- 当前线程拥有其他线程需要的资源
- 当前线程等待其他线程已拥有的资源
- 都不放弃自己拥有的资源

## 3. 如何避免死锁

死锁是我们非常不愿意看到的一种现象，我们要尽可能避免死锁的情况发生。通过设置某些限制条件，去破坏产生死锁的四个必要条件中的一个或者几个，来预防发生死锁。预防死锁是一种较易实现的方法。但是由于所施加的限制条件往往太严格，可能会导致系统资源利用率。

## (八) python线程GIL

### 1. python线程的GIL问题（全局解释器锁）

什么是GIL：由于python解释器设计中加入了解释器锁，导致python解释器同一时刻只能解释执行一个线程，大大降低了线程的执行效率。

导致后果：因为遇到阻塞时线程会主动让出解释器，去解释其他线程。所以python多线程在执行多阻塞高延迟IO时可以提升程序效率，其他情况并不能对效率有所提升。

#### GIL问题建议

- 尽量使用进程完成无阻塞的并发行为
- 不使用c作为解释器（Java C#）

2. 结论：在无阻塞状态下，多线程程序和单线程程序执行效率几乎差不多，甚至还不如单线程效率。但是多进程运行相同内容却可以有明显的效率提升。

## (九) 进程线程的区别联系

### • 区别联系

1. 两者都是多任务编程方式，都能使用计算机多核资源
2. 进程的创建删除消耗的计算机资源比线程多
3. 进程空间独立，数据互不干扰，有专门通信方法；线程使用全局变量通信
4. 一个进程可以有多个分支线程，两者有包含关系
5. 多个线程共享进程资源，在共享资源操作时往往需要同步互斥处理
6. 进程线程在系统中都有自己的特有属性标志，如ID,代码段，命令集等。

### • 使用场景

1. 任务场景：如果是相对独立的任务模块，可能使用多进程，如果是多个分支共同形成一个整体任务可能用多线程
2. 项目结构：多种编程语言实现不同任务模块，可能是多进程，或者前后端分离应该各自为一个进程。
3. 难易程度：通信难度，数据处理的复杂度来判断用进程间通信还是同步互斥方法。

### • 要求

1. 对进程线程怎么理解/说说进程线程的差异
2. 进程间通信知道哪些，有什么特点
3. 什么是同步互斥，你什么情况下使用，怎么用
4. 给一个情形，说说用进程还是线程，为什么
5. 问一些概念，僵尸进程的处理，GIL问题，进程状态

## (十) 并发网络通信模型

### 1. 常见网络模型

1. 循环服务器模型：循环接收客户端请求，处理请求。同一时刻只能处理一个请求，处理完毕后再处理下一个。

优点：实现简单，占用资源少

缺点：无法同时处理多个客户端请求

适用情况：处理的任務可以很快完成，客户端无需长期占用服务端程序。udp比tcp更适合循环。

2. 多进程/线程网络并发模型：每当一个客户端连接服务器，就创建一个新的进程/线程为该客户端服务，客户端退出时再销毁该进程/线程。

优点：能同时满足多个客户端长期占有服务端需求，可以处理各种请求。

缺点：资源消耗较大

适用情况：客户端同时连接量较少，需要处理行为较复杂情况。

3. IO并发模型：利用IO多路复用,异步IO等技术，同时处理多个客户端IO请求。

优点：资源消耗少，能同时高效处理多个IO行为

缺点：只能处理并发产生的IO事件，无法处理cpu计算

适用情况：HTTP请求，网络传输等都是IO行为。

### 2. 基于fork的多进程网络并发模型

代码实现: `day10/fork_server.py`

实现步骤

1. 创建监听套接字
2. 等待接收客户端请求
3. 客户端连接创建新的进程处理客户端请求
4. 原进程继续等待其他客户端连接
5. 如果客户端退出，则销毁对应的进程

### 3. 基于threading的多线程网络并发

实现步骤

1. 创建监听套接字
2. 循环接收客户端连接请求
3. 当有新的客户端连接创建线程处理客户端请求
4. 主线程继续等待其他客户端连接
5. 当客户端退出，则对应分支线程退出

### 4. ftp 文件服务器

代码实现: `day11/ftp`



## 1. 功能

- 【1】 分为服务端和客户端，要求可以有多个客户端同时操作。
- 【2】 客户端可以查看服务器文件库中有什么文件。
- 【3】 客户端可以从文件库中下载文件到本地。
- 【4】 客户端可以上传一个本地文件到文件库。
- 【5】 使用print在客户端打印命令输入提示，引导操作

# (十一) IO并发

## 1.IO 分类

IO分类：阻塞IO，非阻塞IO，IO多路复用，异步IO等

## 2.阻塞IO

1.定义：在执行IO操作时如果执行条件不满足则阻塞。阻塞IO是IO的默认形态。

2.效率：阻塞IO是效率很低的一种IO。但是由于逻辑简单所以是默认IO行为。

3.阻塞情况：

- 因为某种执行条件没有满足造成的函数阻塞  
e.g. `accept input recv`
- 处理IO的时间较长产生的阻塞状态  
e.g. 网络传输，大文件读写

- 非阻塞IO

1. 定义：通过修改IO属性行为，使原本阻塞的IO变为非阻塞的状态。

- 设置套接字为非阻塞IO

```
sockfd.setblocking(bool)
```

功能：设置套接字为非阻塞IO

参数：默认为True，表示套接字IO阻塞；设置为False则套接字IO变为非阻塞

- 超时检测：设置一个最长阻塞时间，超过该时间后则不再阻塞等待。

```
sockfd.settimeout(sec)
```

功能：设置套接字的超时时间

参数：设置的时间

## 3.IO多路复用

### 1. 定义

同时监控多个IO事件，当哪个IO事件准备就绪就执行哪个IO事件。以此形成可以同时处理多个IO的行为，避免一个IO阻塞造成其他IO均无法执行，提高了IO执行效率。

### 2. 具体方案

select方法： windows linux unix

poll方法： linux unix

epoll方法： linux

## select 方法

代码实现: `day11/select_server.py`

```
rs, ws, xs=select(rlist, wlist, xlist[, timeout])
```

功能: 监控IO事件, 阻塞等待IO发生

参数: `rlist` 列表 存放关注的等待发生的IO事件

`wlist` 列表 存放关注的要主动处理的IO事件

`xlist` 列表 存放关注的出现异常要处理的IO

`timeout` 超时时间

返回值: `rs` 列表 `rlist`中准备就绪的IO

`ws` 列表 `wlist`中准备就绪的IO

`xs` 列表 `xlist`中准备就绪的IO

## select 实现tcp服务

- 【1】 将关注的IO放入对应的监控类别列表
- 【2】 通过select函数进行监控
- 【3】 遍历select返回值列表, 确定就绪IO事件
- 【4】 处理发生的IO事件

### 注意

`wlist`中如果存在IO事件, 则select立即返回给ws  
处理IO过程中不要出现死循环占有服务端的情况  
IO多路复用消耗资源较少, 效率较高

## ###@@扩展: 位运算

定义: 将整数转换为二进制, 按二进制位进行运算

运算符号:

& 按位与  
| 按位或  
^ 按位异或  
<< 左移  
>> 右移

e.g. 14 --> 01110  
19 --> 10011

14 & 19 = 00010 = 2 一0则0  
14 | 19 = 11111 = 31 一1则1  
14 ^ 19 = 11101 = 29 相同为0不同为1  
14 << 2 = 111000 = 56 向左移动低位补0  
14 >> 2 = 11 = 3 向右移动去掉低位

## poll方法

代码实现: *day12/poll\_server.py*

```
p = select.poll()
```

功能：创建poll对象

返回值：poll对象

```
p.register(fd,event)
```

功能：注册关注的IO事件

参数：fd 要关注的IO

event 要关注的IO事件类型

常用类型：POLLIN 读IO事件 (rlist)

POLLOUT 写IO事件 (wlist)

POLLERR 异常IO (xlist)

POLLHUP 断开连接

e.g. p.register(sockfd,POLLIN|POLLERR)

```
p.unregister(fd)
```

功能：取消对IO的关注

参数：IO对象或者IO对象的fileno

```
events = p.poll()
```

功能：阻塞等待监控的IO事件发生

返回值：返回发生的IO

events格式 [(fileno,event),(),...]

每个元组为一个就绪IO，元组第一项是该IO的fileno，第二项为该IO就绪的事件类型

## poll\_server 步骤

- 【1】 创建套接字
- 【2】 将套接字register
- 【3】 创建查找字典，并维护
- 【4】 循环监控IO发生
- 【5】 处理发生的IO

## epoll方法

代码实现: *day12/epoll\_server.py*

1. 使用方法：基本与poll相同

- 生成对象改为epoll()
- 将所有事件类型改为EPOLL类型

2. epoll特点

- epoll 效率比select poll要高
- epoll 监控IO数量比select要多
- epoll 的触发方式比poll要多 (EPOLLET边缘触发)

## 4.协程技术

### 基础概念

1. 定义：纤程，微线程。是允许在不同入口点不同位置暂停或开始的计算机程序，简单来说，协程就是可以暂停执行的函数。
2. 协程原理：记录一个函数的上下文，协程调度切换时会将记录的上下文保存，在切换回来时进行调取，恢复原有的执行内容，以便从上一次执行位置继续执行。
3. 协程优缺点

#### 优点

1. 协程完成多任务占用计算资源很少
2. 由于协程的多任务切换在应用层完成，因此切换开销少
3. 协程为单线程程序，无需进行共享资源同步互斥处理

#### 缺点

协程的本质是一个单线程，无法利用计算机多核资源

### ####扩展延伸@标准库协程的实现

python3.5以后，使用标准库asyncio和async/await 语法来编写并发代码。asyncio库通过对异步IO行为的支持完成python的协程。虽然官方说asyncio是未来的开发方向，但是由于其生态不够丰富，大量的客户端不支持awaitable需要自己去封装，所以在使用上存在缺陷。更多时候只能使用已有的异步库（asyncio等），功能有限

### 第三方协程模

1. greenlet模块

#### 示例代码: day12/greenlet\_0.py

- 安装：sudo pip3 install greenlet
- 函数

```
greenlet.greenlet(func)
```

功能：创建协程对象

参数：协程函数

```
g.switch()
```

功能：选择要执行的协程函数

2. gevent模块

#### 示例代码: day12/gevent\_test.py

#### 示例代码: day12/gevent\_server.py

- 安装：sudo pip3 install gevent
- 函数

```
gevent.spawn(func, argv)
```

功能：生成协程对象

参数：**func** 协程函数  
**argv** 给协程函数传参（不定参）

返回值：协程对象

**gevent.joinall(list,[timeout])**

功能：阻塞等待协程执行完毕

参数：**list** 协程对象列表  
**timeout** 超时时间

**gevent.sleep(sec)**

功能：**gevent**睡眠阻塞

参数：睡眠时间

\* **gevent**协程只有在遇到**gevent**指定的阻塞行为时才会自动在协程之间进行跳转  
如**gevent.joinall()**,**gevent.sleep()**带来的阻塞

- **monkey脚本**

作用：在**gevent**协程中，协程只有遇到**gevent**指定类型的阻塞才能跳转到其他协程，因此，我们希望将普通的IO阻塞行为转换为可以触发**gevent**协程跳转的阻塞，以提高执行效率。

转换方法：**gevent** 提供了一个脚本程序**monkey**,可以修改底层解释IO阻塞的行为，将很多普通阻塞转换为**gevent**阻塞。

使用方法

【1】导入**monkey**

```
from gevent import monkey
```

【2】运行相应的脚本，例如转换socket中所有阻塞

```
monkey.patch_socket()
```

【3】如果将所有可转换的IO阻塞全部转换则运行all

```
monkey.patch_all()
```

【4】注意：脚本运行函数需要在对应模块导入前执行

## 5.HTTPServer v2.0

### 1. 主要功能：

- 【1】 接收客户端（浏览器）请求
- 【2】 解析客户端发送的请求
- 【3】 根据请求组织数据内容
- 【4】 将数据内容形成http响应格式返回给浏览器

### 2. 升级点：

- 【1】 采用IO并发，可以满足多个客户端同时发起请求情况
- 【2】 做基本的请求解析，根据具体请求返回具体内容，同时满足客户端简单的非网页请求情况

- 【3】 通过类接口形式进行功能封装

## 十、软件开发的目录规范

### 项目文件夹

```
bin #存放可执行文件
|
| start.py
|
conf #项目的配置文件 如数据库路径、日志路径
|
| settings.py
|
db #数据库文件夹
|
lib #共享库，公用区 common.py
|
core #核心代码逻辑 core.py
|
log #日志文件夹
|
api #
|
| api.py
|
run.py
|
setup.py
|
requirements.txt
|
README
```

注：

- core/:存放业务逻辑相关代码
- api/:存放接口文件，接口主要用于为业务逻辑提供数据操作。
- db/:存放操作数据库相关文件，主要用于与数据库交互
- lib/t:存放程序中常用的自定义模块
- conf:存放配置文件

- `run.py`:程序的启动文件，一般放在项目的根目录下，因为在运行时会默认将运行文件所在的文件夹作为`sys.path`的第一个路径，这样就省去了处理环境变量的步骤
- `.setup.py`:安装、部署、打包的脚本。
- `requirements.txt`:存放软件依赖的外部Python包列表。
- `README`:项目说明文件。